

Univerzitet u Beogradu
Fakultet organizacionih nauka
Katedra za elektronsko poslovanje

Aplikacija za upravljanje finansijama i budžetom

Rbr	Ime	Prezime	Broj indeksa
1.	Jovan	Janjušević	2021/0172
2.	Miloš	Purić	2021/0100
Mentor		Petar Lukovac	
GitHub link		https://github.com/elab-development/internet-tehnologije-2025-smartbudget_2021_0100.git	

Sadržaj

1.	Korisnički zahtev	4
1.1.	Verbalni opis za izabranu temu.....	4
1.2.	Funkcionalni zahtevi.....	5
1.3.	Nefunkcionalni zahtevi	7
1.4.	Uloge nad sistemom i permisije	9
2.	Opis sistema	10
2.1.	Opis slučaja korišćenja (Use Case opis)	10
	SK1 – Registracija korisnika	12
	SK2 – Prijava korisnika (Login)	13
	SK3 – Odjava sa sistema	14
	SK4 – Kreiranje novčanika (Wallet)	15
	SK5 – Izmena novčanika	16
	SK6 – Brisanje novčanika	17
	SK7 – Prenos sredstava (Transfer)	18
	SK8 – Unos transakcije	19
	SK9 – Filtriranje transakcija	20
	SK10 – Brisanje transakcije	21
	SK11 – Postavljanje mesečnog budžeta	22
	SK12 – Brisanje budžeta.....	23
	SK13 – Pregled izveštaja (Dashboard)	24
	SK14 – Pregled svih korisnika (ADMIN)	25
	SK15 – Blokiranje korisnika	26
	SK16 – Dodavanje sistemske kategorije	27
	SK17 – Izmena sistemske kategorije	28
	SK18 – Brisanje sistemske kategorije	29
2.2.	Arhitektura aplikacije.....	30
2.3.	Opis procesa slučaja korišćenja (Dijagrami sekvenci)	32
2.4.	Model podataka – PMOV ili Dijagram klasa	36

3.	Predlog tehnologija koje će biti korišćene.....	37
3.1.	Frontend.....	37
3.2.	Backend	37
3.3.	Baza podataka	38
3.4.	Integracije.....	38
3.5.	DevOps	38
4.	Prikaz implementacije i korisničkog interfejsa	39
4.1.	Prikaz delova koda sa objašnjenjima.....	39
4.2.	Prikaz korisničkog interfejsa (Frontend UI)	44
4.3.	Povezanost zahteva i implementacije	56
5.	Tehnička realizacija i DevOps postavka projekta	58
5.1.	Dokerizacija aplikacije.....	58
5.2.	API specifikacija (Swagger)	59
5.3.	Integracija sa eksternim API servisima	60
5.4.	Dokumentacija projekta (README).....	61
5.5.	Dokumentacija projekta (README).....	62
5.6.	CI/CD pipeline	63
5.7.	Postavljanje aplikacije na Cloud	65
5.8.	Bezbednost aplikacije.....	66
5.9.	Automatizovani testovi	67
5.10.	Vizuelizacija podataka	69

1. Korisnički zahtev

1.1. Verbalni opis za izabranu temu

Kratak uvod u problem i motivaciju

U savremenom digitalnom dobu, upravljanje ličnim finansijama predstavlja jedan od ključnih izazova za pojedince i domaćinstva. Veliki broj ljudi, posebno studenata i mladih profesionalaca, suočava se sa problemom nedostatka uvida u tokovima svog novca. Impulsivna potrošnja, nepostojanje evidencije o sitnim troškovima i nedostatak planiranja budžeta često dovode do finansijske nestabilnosti. Tradicionalni načini vođenja evidencije (papir, Excel tabele) su često nepraktični za svakodnevnu upotrebu u pokretu, što dovodi do odustajanja od praćenja troškova.

Opis ciljeva aplikacije

Osnovni cilj aplikacije *SmartBudget* je da digitalizuje i pojednostavi proces upravljanja ličnim finansijama. Aplikacija ima za cilj da korisnicima pruži centralizovano mesto za evidentiranje svih finansijskih transakcija, omogućavajući im jasan i trenutni uvid u njihovo finansijsko stanje. Pored puke evidencije, cilj sistema je da edukuje korisnika o njegovim navikama potrošnje kroz vizuelne izveštaje i pomogne mu da ostane u okvirima definisanog budžeta.

Opis ciljne grupe korisnika

Sistem je namenjen širokom spektru korisnika koji žele da unaprede svoju finansijsku pismenost, a primarno se izdvajaju tri grupe:

- **Studenti:** Korisnici koji raspolažu ograničenim sredstvima (džeparac, stipendija) i imaju potrebu za strogom kontrolom troškova života i studiranja.
- **Zaposleni pojedinci:** Osobe koje žele da prate odnos svojih prihoda i rashoda, planiraju štednju ili otplatu kredita.
- **Freelancer-i:** Korisnici kojima je potrebno da na jednostavan način razdvoje i prate personalne troškove u odnosu na poslovne prihode.

Sažet pregled funkcionalnosti koje aplikacija omogućava

Veb aplikacija omogućava korisnicima kreiranje i personalizaciju naloga, nakon čega mogu unositi prihode i rashode uz definisanje kategorija (hrana, računi, zabava). Sistem omogućava postavljanje mesečnih limita potrošnje po kategorijama i automatski obaveštava korisnika o statusu budžeta. Pored toga, aplikacija generiše grafičke izveštaje (statistike) koji prikazuju strukturu potrošnje u određenom vremenskom periodu, olakšavajući analizu i donošenje odluka.

1.2. Funkcionalni zahtevi

Na osnovu analize potreba korisnika, definisani su sledeći funkcionalni zahtevi (FZ), grupisani po logičkim celinama:

1. Korisnički nalozi i autentifikacija

- **Registracija korisnika:** Aplikacija mora omogućiti kreiranje korisničkog naloga putem unosa osnovnih podataka (ime, email, lozinka). Sistem mora proveriti jedinstvenost email adrese i validirati složenost lozinke pre kreiranja zapisa u bazi.
- **Prijava korisnika (Login):** Korisnik mora moći da se prijavi na sistem unosom ispravnog email-a i lozinke. Sistem mora generisati bezbednosni autentifikacioni token (JWT) koji omogućava pristup zaštićenim delovima aplikacije.
- **Odjava korisnika:** Korisnik mora imati mogućnost da se bezbedno odjavi iz sistema, pri čemu klijentska aplikacija briše sačuvani token, a sesija postaje nevažeća.

2. Upravljanje finansijskim sredstvima i transakcijama

- **Upravljanje računima (Wallets):** Korisnik mora imati mogućnost definisanja izvora plaćanja (npr. Keš, Tekući račun, Štednja) i pregleda trenutnog stanja na svakom od njih.
- **Kreiranje transakcije:** Korisnik mora imati mogućnost unosa nove transakcije (prihod ili rashod) sa obaveznim atributima: iznos, tip transakcije, kategorija, datum i račun (Wallet) sa kog se plaća, te opcionim opisom.
- **Pregled transakcija:** Korisnik mora imati uvid u listu svih svojih transakcija, hronološki sortiranih. Sistem mora obezbediti mehanizme filtriranja (po kategoriji, tipu, vremenskom periodu ili računu).
- **Izmena transakcije:** Korisnik mora imati mogućnost naknadne izmene detalja postojeće transakcije (npr. ispravka iznosa ili promene kategorije), uz automatsko ažuriranje salda.
- **Brisanje transakcije:** Korisnik mora moći da obriše pogrešno unetu transakciju, što mora rezultirati vraćanjem sredstava na odgovarajući saldo.

3. Kategorizacija troškova

- **Pregled kategorija:** Korisnik mora imati mogućnost pregleda liste svih raspoloživih kategorija za klasifikaciju troškova (kombinacija sistemskih i korisničkih kategorija).
- **Administratorsko upravljanje kategorijama:** Administrator sistema mora imati mogućnost kreiranja novih globalnih kategorija, izmene naziva postojećih i brisanja kategorija koje nisu u upotrebi, čime se obezbeđuje konzistentnost šifarnika.

4. Budžet i kontrola potrošnje

- **Postavljanje budžeta:** Korisnik mora imati mogućnost definisanja mesečnog limita potrošnje, bilo na nivou ukupnog budžeta ili za specifične kategorije (npr. limit za izlaske).
- **Praćenje realizacije budžeta:** Sistem mora automatski pratiti i izračunavati trenutnu potrošnju u odnosu na postavljeni limit i prikazivati preostali raspoloživi iznos.
- **Obaveštenje o statusu budžeta:** Sistem mora vizuelno obavestiti korisnika, internom notifikacijom, kada se potrošnja približi limitu (npr. na 90%) ili kada dođe do prekoračenja budžeta.

5. Izveštavanje i analitika

- **Grafički izveštaji:** Sistem mora generisati vizuelne prikaze podataka (grafikone) koji ilustruju odnos prihoda i rashoda, kao i procentualnu raspodelu troškova po kategorijama (Pie chart).

6. Administratorske funkcionalnosti

- **Pregled korisnika:** Administrator mora imati uvid u listu svih registrovanih korisnika u sistemu radi monitoringa.
- **Upravljanje pristupom:** Administrator mora imati mogućnost da deaktivira (blokira) ili reaktivira korisnički nalog u slučaju kršenja pravila korišćenja ili na zahtev korisnika.

1.3. Nefunkcionalni zahtevi

Nefunkcionalni zahtevi definišu attribute kvaliteta, performanse i ograničenja sistema, osiguravajući da aplikacija ne samo radi ono što treba, već i da to radi na kvalitetan i pouzdan način.

1. Performanse

- **Vreme odziva:** Sistem mora obezbediti vreme odziva servera (latency) manje od 500ms za standardne CRUD operacije pri normalnom opterećenju.
- **Brzina učitavanja:** Inicijalno učitavanje korisničkog interfejsa (Frontend rendering) treba da bude izvršeno za manje od 2 sekunde na prosečnoj internet konekciji (4G/Broadband).
- **Konkurentnost:** Sistem treba da podrži rad više korisnika istovremeno bez degradacije performansi, uz efikasno upravljanje konekcijama ka bazi podataka.

2. Bezbednost

- **Autentifikacija i Autorizacija:** Pristup zaštićenim resursima mora biti omogućen isključivo autentifikovanim korisnicima putem JWT (JSON Web Token) mehanizma.
- **Zaštita podataka:** Lozinke korisnika se ne smeju čuvati u čitljivom formatu, već moraju biti heširane korišćenjem jakih algoritama (npr. bcrypt).
- **Sigurna komunikacija:** Sva komunikacija između klijenta i servera u produkcionom okruženju mora se odvijati putem HTTPS protokola radi zaštite od presretanja podataka (Man-in-the-Middle napadi).
- **Izolacija podataka:** Sistem mora striktno sprovoditi kontrolu pristupa tako da jedan korisnik ne može pristupiti podacima drugog korisnika (zaštita od IDOR ranjivosti).

3. Pouzdanost i dostupnost

- **Validacija podataka:** Sistem mora vršiti strogu validaciju svih ulaznih podataka na serverskoj strani (Backend) kako bi se sprečio unos nekonzistentnih podataka.
- **Integritet transakcija:** Finansijske transakcije moraju biti atomske – ili su u potpunosti izvršene ili nisu uopšte, čime se sprečava delimično ažuriranje stanja.
- **Rukovanje greškama:** U slučaju greške, sistem mora korisniku prikazati razumljivu poruku ("User-friendly error message") umesto tehničkih detalja koda.

4. Upotrebljivost (UI/UX)

- **Jezik i lokalizacija:** Korisnički interfejs mora biti na srpskom jeziku (latinica), sa formatima datuma i valuta prilagođenim domaćem tržištu (RSD, dd.mm.yyyy.).
- **Dizajn:** Interfejs treba da bude intuitivan, minimalistički i jednostavan za navigaciju, sa jasnim vizuelnim povratnim informacijama za akcije korisnika).
- **Korisničko iskustvo:** Broj koraka (klikova) potrebnih za unos transakcije treba da bude minimalan.

5. Održavanje i skalabilnost

- **Modularnost:** Arhitektura aplikacije mora biti modularna (razdvojen Frontend, Backend i Baza), što omogućava nezavisan razvoj i lakše održavanje delova sistema.
- **Čitljivost koda:** Izvorni kod mora pratiti standarde i konvencije izabranih tehnologija, uz adekvatne komentare za kompleksnu poslovnu logiku.
- **Proširivost:** Sistem mora biti dizajniran tako da omogućava lako dodavanje novih funkcionalnosti u budućnosti (npr. dodavanje novih tipova izveštaja ili povezivanje sa eksternim bankarskim API-jima).

6. Portabilnost

- **Responzivnost (Responsive Design):** Aplikacija mora biti u potpunosti prilagođena prikazu na različitim uređajima (mobilni telefoni, tableti, desktop računari).
- **Podrška za pretraživače:** Aplikacija mora ispravno funkcionisati na svim modernim veb pretraživačima (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge).

1.4. Uloge nad sistemom i permisije

<i>Uloga</i>	<i>Dozvoljene funkcije</i>	<i>Ograničenja</i>	<i>Prava pristupa podacima</i>
<i>Neregistrovani korisnik (Guest)</i>	• Pregled početne (Landing) stranice • Registracija novog naloga • Prijava na sistem (Login)	• Nema pristup funkcionalnostima aplikacije (Dashboard, Transakcije) • Ne može videti podatke drugih korisnika	Nema prava pristupa. Ne može čitati niti upisivati podatke u bazu osim prilikom procesa registracije.
<i>Registrovani korisnik (User)</i>	• Upravljanje finansijama: Kreiranje, čitanje, izmena i brisanje (CRUD) transakcija i novčanika. • Budžetiranje: Definisanje limita potrošnje. • Analitika: Pregled ličnih grafikona i izveštaja.	• Izolacija: Striktno mu je zabranjen pristup podacima drugih korisnika. • Admin panel: Nema pristup administrativnim funkcijama. • Globalne postavke: Ne može menjati sistemske kategorije.	Pristup sopstvenim podacima (Owner only). Ima pravo čitanja i upisa (<i>SELECT</i> , <i>INSERT</i> , <i>UPDATE</i> , <i>DELETE</i>) nad tabelama <i>Transactions</i> , <i>Wallets</i> , <i>Budgets</i> gde je <i>user_id</i> jednak njegovom ID-u.
<i>Administrator</i>	• Upravljanje korisnicima: Pregled liste korisnika, deaktivacija (banovanje) sumnjivih naloga. • Sistemske šifarnici: Dodavanje i brisanje globalnih kategorija transakcija.fu	• Privatnost: Ne može (i ne sme) da vidi detalje pojedinačnih finansijskih transakcija korisnika, stanja na računima niti iznose budžeta. • Finansije: Ne može da kreira transakcije u ime drugih korisnika.	Sistemske pristup. Ima pravo čitanja (<i>SELECT</i>) nad tabelom <i>Users</i> (meta-podaci). Ima puno pravo (CRUD) nad tabelom <i>Categories</i> (za globalne kategorije). Nema pravo pristupa tabeli <i>Transactions</i> .

Tabela 1 - Uloge nad sistemom i permisije

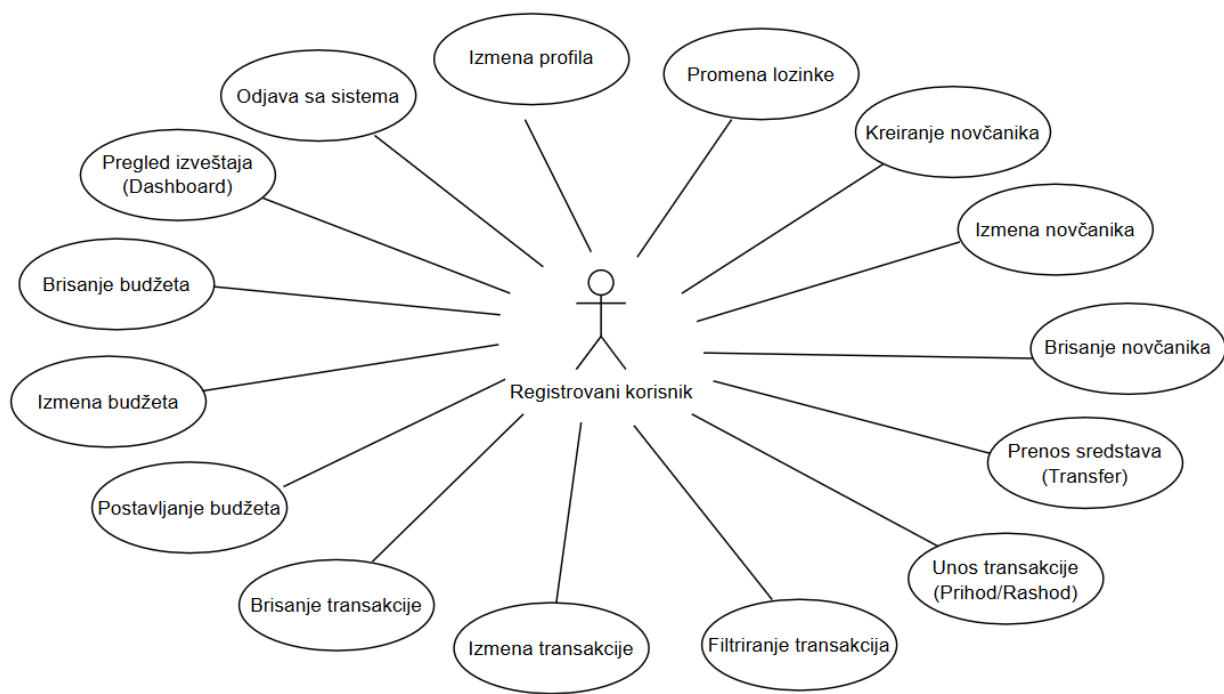
2. Opis sistema

2.1. Opis slučajeve korišćenja (Use Case opis)

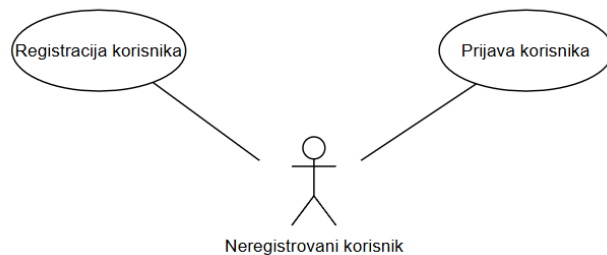
Šifra SK	Naziv slučaja korišćenja	Preduslovi
SK1	Registracija korisnika	/
SK2	Prijava korisnika (Login)	Korisnički nalog je kreiran
SK3	Odjava sa sistema	Korisnik je prijavljen
SK4	Kreiranje novčanika	Korisnik je prijavljen
SK5	Izmena novčanika	Prijavljen, Lista novčanika
SK6	Brisanje novčanika	Prijavljen, Lista novčanika
SK7	Prenos sredstava (Transfer)	Prijavljen, Postoje bar 2 novčanika
SK8	Unos transakcije	Prijavljen, Liste: Kategorije, Novčanici
SK9	Filtriranje transakcije	Korisnik je prijavljen
SK10	Brisanje transakcije	Prijavljen, Lista transakcija
SK11	Postavljanje mesečnog budžeta	Prijavljen, Lista kategorija
SK12	Brisanje budžeta	Prijavljen, Lista budžeta
SK13	Pregled izveštaja (Dashboard)	Korisnik je prijavljen
SK14	Pregled svih korisnika	Admin je prijavljen
SK15	Blokiranje	Admin je prijavljen
SK16	Dodavanje sistemske kategorije	Admin je prijavljen
SK17	Izmena sistemske kategorije	Admin je prijavljen, Lista kategorija
SK18	Brisanje sistemske kategorije	Admin je prijavljen, Lista kategorija

Tabela 2 - Kompletna lista slučajeve korišćenja

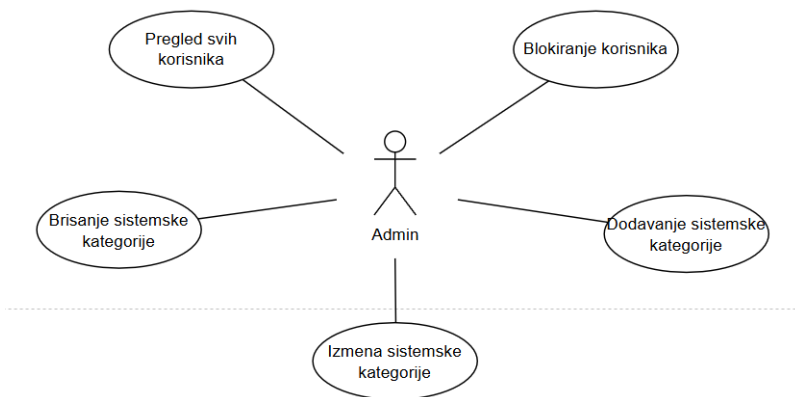
Na sledećoj strani prikazani su dijagrami slučajeve korišćenja.



Slika 1 - Slučajevi korišćenja registrovanog korisnika



Slika 2 - Slučajevi korišćenja neregistrovanog korisnika



Slika 3 - Slučajevi korišćenja admina

U nastavku su detaljno opisani slučajevi korišćenja.

SK1 – Registracija korisnika

Glavne uloge (akteri): Neregistrovani korisnik

Kratak opis: Slučaj opisuje proces kreiranja novog korisničkog naloga unosom osnovnih podataka i čuvanjem zapisa u bazi.

Preduslovi:

- Korisnik nije prijavljen.
- Forma za registraciju je učitana.

Glavni tok:

1. Korisnik unosi ime, email i lozinku u polja forme. (APUSO)
2. Korisnik potvrđuje registraciju klikom na dugme "Registruj se". (APSO)
3. Sistem proverava validnost podataka (format email-a, jačina lozinke). (SO)
4. Sistem proverava da li email već postoji u bazi. (SO)
5. Sistem kreira novi korisnički nalog i upisuje podatke u bazu. (SO)
6. Sistem prikazuje poruku: „Uspešno ste registrovani“. (IA)

Alternativni tokovi:

- 3.1. Sistem prikazuje poruku o grešci: „Lozinka mora imati najmanje 8 karaktera.“ (IA)
- 4.1. Sistem prikazuje poruku: „Nalog sa ovom email adresom već postoji.“ (IA)

Postuslovi:

- Novi korisnik je kreiran u bazi podataka.
- Korisnik je spreman za prijavu na sistem.

SK2 – Prijava korisnika (Login)

Glavne uloge (akteri): Neregistrovani korisnik

Kratak opis: Korisnik pristupa sistemu unošenjem kredencijala kako bi pokrenuo zaštićenu sesiju.

Preduslovi:

- Korisnik je registrovan.
- Forma za prijavu je učitana.

Glavni tok:

1. Korisnik unosi email i lozinku. (APUSO)
2. Korisnik potvrđuje unos klikom na dugme *Prijavi se*. (APSO)
3. Sistem proverava da li korisnik postoji i da li je lozinka tačna. (SO)
4. Sistem generiše pristupni token (JWT). (SO)
5. Sistem preusmerava korisnika na Dashboard. (IA)

Alternativni tokovi:

- 3.1 Sistem prikazuje poruku: „Pogrešna email ili lozinka“. (IA)

Postuslovi:

- Korisnik je ulogovan i sesija je aktivna (sa sačuvanim JWT tokenom).

SK3 – Odjava sa sistema

Glavne uloge (akteri): Registrovani korisnik / Administrator

Kratak opis: Korisnik završava rad sa aplikacijom i sistem uništava sesiju.

Preduslovi:

- Korisnik je prijavljen.

Glavni tok:

1. Korisnik klikne na dugme *Odjavi se*. (APSO)
2. Sistem briše sačuvani pristupni token iz memorije pretraživača. (SO)
3. Sistem preusmerava korisnika na stranicu za prijavu. (IA)

Postuslovi:

- Korisnik više nema pristup zaštićenim resursima.

SK4 – Kreiranje novčanika (Wallet)

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik kreira novi novčanik (Wallet) u koji može unositi transakcije (keš, tekući račun, kartica i sl.).

Preduslovi:

- Korisnik je prijavljen.
- Stranica „Novčanici“ je učitana.

Glavni tok:

1. Korisnik bira opciju „Dodaj novčanik“. (APSO)
2. Sistem prikazuje formu za unos naziva i valute novčanika. (IA)
3. Korisnik unosi podatke. (APSO)
4. Sistem validira naziv i valutu. (SO)
5. Sistem upisuje novi novčanik u bazu i dodeljuje ga korisniku. (SO)
6. Sistem prikazuje poruku: „Novčanik uspešno kreiran.“ (IA)

Alternativni tokovi:

- 4.1. Sistem obaveštava korisnika da ne može imati dva novčanika sa istim nazivom. (IA)
- 4.2. Sistem prikazuje: „Odabrana valuta nije dostupna.“ (IA)

Postuslovi:

- Novi novčanik se pojavljuje u listi Wallet-a korisnika.
- Spreman je za unos transakcija.

SK5 – Izmena novčanika

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik menja naziv postojećeg novčanika i opcionalno valutu.

Preduslovi:

- Korisnik je prijavljen.
- Učitana je lista svih novčanika

Glavni tok:

1. Korisnik otvara formu za izmenu i menja podatke o novčaniku. (APUSO)
2. Korisnik potvrđuje izmenu klikom na *Sačuvaj*. (APSO)
3. Sistem validira izmenjene podatke. (SO)
4. Sistem ažurira podatke o odabranom novčaniku. (SO)
5. Sistem osvežava prikaz liste novčanika. (IA)

Alternativni tokovi:

- 3.1. Sistem prikazuje grešku: „Novčanik sa ovim nazivom već postoji“. (IA)

Postuslovi:

- Novčanik ima nove podatke.

SK6 – Brisanje novčanika

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik trajno uklanja novčanik i sve povezane transakcije.

Preduslovi:

- Korisnik je prijavljen.
- Učitana je lista svih novčanika

Glavni tok:

1. Korisnik bira opciju *Obriši* pored željenog novčanika. (APSO)
2. Sistem prikazuje upozorenje za potvrdu brisanja i obaveštava da će sve povezane transakcije biti obrisane. (IA)
3. Korisnik potvrđuje brisanje. (APSO)
4. Sistem briše novčanik i sve njegove transakcije iz baze. (SO)
5. Sistem osvežava prikaz i šalje poruku o uspehu. (IA)

Postuslovi:

- Novčanik i svi njegovi podaci su trajno uklonjeni.

SK7 – Prenos sredstava (Transfer)

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik prebacuje novac sa jednog novčanika na drugi, bez promene ukupnog salda.

Preduslovi:

- Korisnik ima bar dva aktivna novčanika.

Glavni tok:

1. Korisnik otvara formu za transfer. (APSO)
2. Korisnik bira Izvorni i Ciljni novčanik, te unosi iznos transfera. (APUSO)
3. Sistem proverava da li na Izvornom novčaniku ima dovoljno sredstava (ako se radi samo o pozitivnim saldima). (SO)
4. Sistem ažurira stanje Izvornog novčanika (umanjuje iznos). (SO)
5. Sistem ažurira stanje Ciljanog novčanika (uvećava iznos). (SO)
6. Sistem evidentira transfer kao transakciju tipa "Transfer". (SO)

Alternativni tokovi:

- 2.1 Sistem prikazuje upozorenje da transfer nije moguć. (IA)
- 3.1 Sistem prikazuje grešku: "Nimate dovoljno sredstava na izvornom računu". (IA)

Postuslovi:

- Stanja na oba novčanika su ažurirana.
- Transakcija transfera je zabeležena.

SK8 – Unos transakcije

Akteri: Registrovani korisnik

Kratak opis: Korisnik unosi finansijsku transakciju (prihod ili rashod), pri čemu se stanje pripadajućeg novčanika automatski ažurira.

Preduslovi:

- Korisnik je prijavljen.
- Postoji bar jedan novčanik.
- Učitane su liste kategorija i Wallet-a.

Glavni tok:

1. Korisnik klikne na *Nova transakcija*. (APSO)
2. Sistem prikazuje formu (iznos, tip, kategorija, Wallet, datum, opis). (IA)
3. Korisnik unosi podatke. (APUSO)
4. Korisnik potvrđuje unos. (APSO)
5. Sistem proverava validnost (da li su popunjena obavezna polja i da li je iznos > 0). (SO)
6. Sistem upisuje transakciju u bazu. (SO)
7. Sistem ažurira stanje Wallet-a (umanjuje za rashod, uvećava za prihod). (SO)
8. Sistem šalje poruku o uspehu. (IA)

Alternativni tokovi:

- 5.1 Sistem prikazuje: „Popunite sva obavezna polja.“ (IA)
- 5.2 Sistem odbija unos i prikazuje grešku. (IA)
- 6.1 Sistem prikazuje poruku o grešci. (IA)

Postuslovi:

- Transakcija je kreirana i vidljiva u listi transakcija.
- Wallet saldo je ispravno ažuriran.

SK9 – Filtriranje transakcija

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik filtrira istoriju transakcija po zadatim kriterijumima (kategorija, period) radi lakšeg uvida.

Preduslovi:

- Korisnik je na stranici *Transakcije* i postoje unete transakcije.
- Učitane su liste dostupnih kategorija i novčanika.

Glavni tok:

1. Korisnik pristupa sekciji za filtriranje. (APSO)
2. Korisnik bira kriterijume iz padajućih menija. (APUSO)
3. Sistem šalje upit Backend-u sa izabranim kriterijumima. (SO)
4. Backend vraća samo transakcije koje zadovoljavaju uslov. (SO)
5. Sistem prikazuje filtriranu listu korisniku. (IA)

Alternativni tokovi:

- 4.1 Sistem prikazuje poruku: „Nema transakcija koje odgovaraju kriterijumu“. (IA)

Postuslovi:

- Prikazani su samo traženi podaci.

SK10 – Brisanje transakcije

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik uklanja pogrešno unetu transakciju iz sistema.

Preduslovi:

- Transakcija postoji.

Glavni tok:

1. Korisnik klikne na *Obriši transakciju*. (APSO)
2. Sistem traži potvrdu za brisanje. (IA)
3. Sistem briše zapis transakcije. (SO)
4. Sistem osvežava prikaz. (IA)

Alternativni tokovi:

- Nema logički relevantnih alternativnih tokova. (IA)

Postuslovi:

- Transakcija je obrisana, a saldo novčanika vraćen na pređašnje stanje.

SK11 – Postavljanje mesečnog budžeta

Akteri: Registrovani korisnik

Kratak opis: Korisnik postavlja limit budžeta za određeni mesec ili kategoriju, kako bi pratio potrošnju.

Preduslovi:

- Korisnik je prijavljen.
- Učitana lista kategorija.

Glavni tok:

1. Korisnik otvara stranicu *Budžet*. (APSO)
2. Korisnik bira kategoriju ili ukupni budžet. (APUSO)
3. Korisnik unosi limit. (APUSO)
4. Sistem validira unos (proverava da li je broj i da li je pozitivan). (SO)
5. Sistem čuva budžetsko ograničenje u bazi. (SO)
6. Sistem prikazuje: „Budžet uspešno postavljen.“ (IA)

Alternativni tokovi:

- 4.1. Sistem prikazuje validacionu grešku. (IA)

Postuslovi:

- Novi budžet je uskladišten.
- Aplikacija sada prati potrošnju prema postavljenom limitu.

SK12 – Brisanje budžeta

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik trajno uklanja budžet.

Preduslovi:

- Budžet je prethodno kreiran.

Glavni tok:

1. Korisnik bira opciju *Obriši*. (APSO)
2. Sistem traži potvrdu za brisanje. (IA)
3. Sistem briše zapis budžeta iz baze podataka. (SO)
4. Sistem osvežava prikaz budžeta. (IA)

Postuslovi:

- Budžet je uklonjen iz sistema.

SK13 – Pregled izveštaja (Dashboard)

Glavne uloge (akteri): Registrovani korisnik

Kratak opis: Korisnik pregleda vizuelne izveštaje, ukupne saldo računa i status budžeta.

Preduslovi:

- Korisnik je prijavljen.

Glavni tok:

1. Korisnik pristupa *Dashboard* stranici. (APSO)
2. Sistem povlači podatke o novčanicima, aktivnim budžetima i transakcijama za tekući period. (SO)
3. Sistem obrađuje podatke (izračunava potrošnju po kategorijama, status budžeta). (SO)
4. Sistem prikazuje grafikone. (IA)

Alternativni tokovi:

- 3.1 Sistem prikazuje poruku: „Nema dovoljno podataka za prikaz u izveštaju”. (IA)

Postuslovi:

- Korisnik ima uvid u svoje finansijsko stanje.

SK14 – Pregled svih korisnika (ADMIN)

Akteri: Administrator

Kratak opis: Administrator pregledava listu svih registrovanih korisnika u cilju nadzora i kontrole.

Preduslovi:

- Admin je prijavljen.

Glavni tok:

1. Admin bira opciju *Korisnici*. (APSO)
2. Sistem preuzima listu korisnika iz baze. (SO)
3. Sistem prikazuje listu korisnika administratoru. (IA)
4. Admin pregleda podatke. (ANSO)

Alternativni tokovi:

- 3.1. Sistem prikazuje poruku: „Nema registrovanih korisnika.“ (IA)

Postuslovi:

- Lista korisnika je bila prikazana.
- Nisu vršene nikakve izmene podataka.

SK15 – Blokiranje korisnika

Glavne uloge (akteri): Administrator

Kratak opis: Administrator privremeno onemogućava pristup sistemu izabranom korisniku.

Preduslovi:

- Admin je prijavljen.
- Korisnik kojeg treba blokirati je trenutno aktivan.
- Učitana je lista svih korisnika.

Glavni tok:

1. Administrator pronalazi željenog korisnika u listi. (ANSO)
2. Bira opciju *Blokiraj* pored izabranog korisnika. (APSO)
3. Sistem traži potvrdu za blokiranje. (IA)
4. Sistem menja status korisnika u blokiran. (SO)
5. Sistem prikazuje poruku o uspehu. (IA)

Alternativni tokovi:

- 2.1. Sistem prikazuje poruku: „Korisnik je već blokiran i ova akcija nije izvršena.“ (IA)

Postuslovi:

- Korisnik je blokiran i ne može se prijaviti.

SK16 – Dodavanje sistemske kategorije

Glavne uloge (akteri): Administrator

Kratak opis: Administrator dodaje novu, globalno dostupnu kategoriju za evidentiranje transakcija.

Preduslovi:

- Administrator je prijavljen sa ulogom Admin.
- Kategorija sa tim imenom ne sme da postoji.

Glavni tok:

1. Admin pristupa stranici za upravljanje kategorijama. (APSO)
2. Admin unosi podatke o novoj kategoriji. (APUSO)
3. Admin potvrđuje unos. (APSO)
4. Sistem proverava da li naziv kategorije već postoji. (SO)
5. Sistem kreira novu sistemska kategoriju. (SO)
6. Sistem prikazuje poruku o uspehu. (IA)

Alternativni tokovi:

- 4.1 Sistem prikazuje poruku o grešci: „Kategorija sa tim nazivom već postoji“. (IA)

Postuslovi:

- Nova sistemska kategorija je kreirana i dostupna svim korisnicima.

SK17 – Izmena systemske kategorije

Glavne uloge (akteri): Administrator

Kratak opis: Administrator menja naziv i/ili tip postojeće systemske kategorije.

Preduslovi:

- Administrator je prijavljen.
- Kategorija je kreirana i vidljiva na listi.
- Učitana lista svih systemskih kategorija.

Glavni tok:

1. Admin bira kategoriju iz liste i otvara formu za izmenu. (APSO)
2. Admin menja željene attribute kategorije. (APUSO)
3. Admin potvrđuje izmene. (APSO)
4. Sistem validira izmene. (SO)
5. Sistem ažurira zapis u bazi. (SO)

Alternativni tokovi:

- 4.1 Sistem prikazuje poruku o grešci. (IA)

Postuslovi:

- Podaci o systemskoj kategoriji su ažurirani, a promene se reflektuju u svim povezanim transakcijama.

SK18 – Brisanje sistemske kategorije

Glavne uloge (akteri): Administrator

Kratak opis: Administrator uklanja sistemsku kategoriju.

Preduslovi:

- Administrator je prijavljen.
- Učitana lista svih sistemskih kategorija.

Glavni tok:

1. Admin bira kategoriju i opciju *Obriši*. (APSO)
2. Sistem proverava da li je kategorija povezana sa bilo kojom aktivnom transakcijom ili budžetom. (SO)
3. Admin potvrđuje brisanje. (APSO)
4. Sistem briše kategoriju iz baze. (SO)

Alternativni tokovi:

2.1 Sistem prikazuje upozorenje: „Kategorija je u upotrebi i ne može se obrisati dok se ne izmene povezane transakcije/budžeti“. (IA)

Postuslovi:

- Kategorija je trajno uklonjena.

2.2. Arhitektura aplikacije

Aplikacija je razvijena korišćenjem **Next.js** tehnologije koja objedinjuje klijentsku i serversku stranu. To znači da smo u okviru jednog repozitorijuma implementirali i React komponente za prikaz i API rute za komunikaciju sa bazom. Ova arhitektura je izabrana jer je idealna za timski rad i omogućava efikasno ispunjenje svih zahteva projekta, uključujući i implementaciju **REST API-ja**.

Arhitektura se sastoji od sledećih ključnih komponenti:

1. Frontend (React.js):

- Realizovan kao skup **React komponenti** unutar Next.js okruženja.
- Koristi **React Hooks** (useState, useEffect, useNavigate) za upravljanje stanjem aplikacije i navigacijom.
- Dizajniran je modularno sa **reusable komponentama** (Dugmad, Polja forme, Kartice, Modali) koje se koriste kroz više stranica.
- Komunikacija sa serverom se odvija asinhrono putem HTTP zahteva ka sopstvenim API rutama.

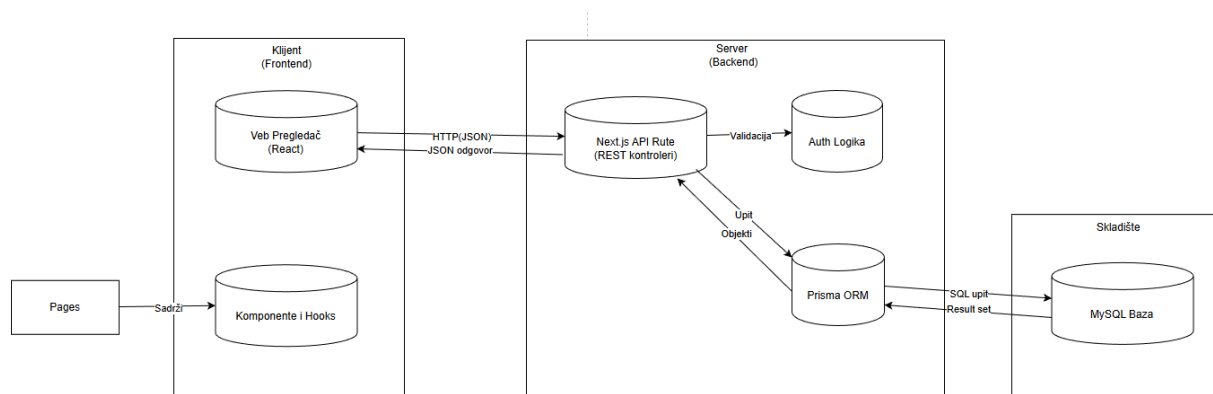
2. Backend (Node.js API):

- Realizovan putem **Next.js API Routes** mehanizma koji funkcioniše po principima **REST API** konvencije.
- Sadrži **Kontrolere** koji obrađuju HTTP zahteve (GET, POST, PUT, DELETE) i vraćaju odgovore u **JSON** formatu.
- Implementira logiku autentifikacije (Login, Register) i štiti rute (Middleware) kako bi pristup izmenama podataka imali samo ulogovani korisnici.

3. Baza podataka (MySQL):

- **Relaciona baza podataka** koja čuva entitete sistema (Korisnici, Transakcije, Budžeti) i definiše njihove međusobne veze putem **spoljnih ključeva** (Foreign Keys).

Tok podataka: Frontend šalje zahtev API ruti. API ruta validira podatke i koristi Prisma klijent da izvrši SQL upit nad **MySQL** bazom. Baza vraća rezultat, a API ga šalje nazad Frontendu kao JSON odgovor.

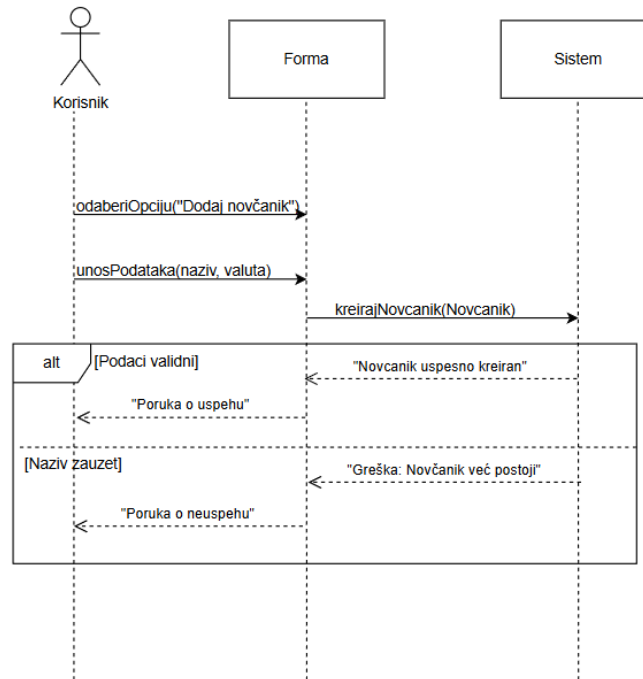


Slika 4 - Tok podataka između komponenti

Komunikacija unutar sistema odvija se kroz definisani ciklus zahteva i odgovora (Request-Response), prateći principe REST arhitekture. Tok podataka kroz slojeve aplikacije izgleda ovako:

1. **Inicijacija na Klijentu:** Korisnik unosi podatke ili zahteva informaciju putem korisničkog interfejsa (React Frontend). Aplikacija prikuplja podatke i pakuje ih u JSON format.
2. **Slanje Zahteva:** Frontend šalje asinhroni HTTP zahtev (GET, POST, PUT ili DELETE) ka odgovarajućoj Next.js API ruti (Backend).
3. **Obrada na Serveru:** API kontroler prima zahtev, vrši validaciju pristiglih podataka i proverava identitet korisnika (Autentifikacija).
4. **Interakcija sa Bazom:** Ukoliko je zahtev validan, Backend koristi Prisma ORM klijent da prevede zahtev u SQL upit i izvrši ga nad MySQL bazom podataka.
5. **Odgovor i Prikaz:** Baza vraća tražene podatke ili potvrdu o uspešnoj akciji. Backend šalje te podatke nazad Frontendu u JSON formatu, nakon čega React ažurira prikaz korisniku bez potrebe za osvežavanjem cele stranice.

2.3. Opis procesa slučajeva korišćenja (Dijagrami sekvenci)

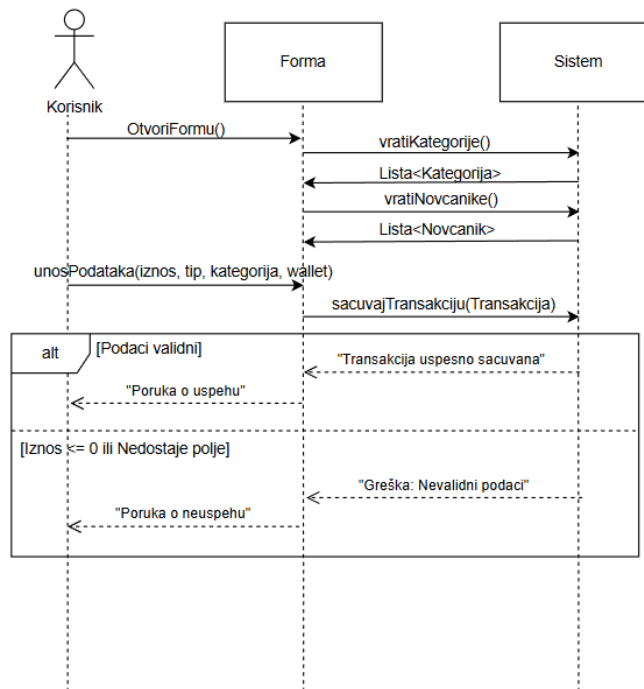


Slika 5 - Dijagram sekvenci za SK4 - Kreiranje novčanika

Prikazani dijagram ilustruje sekvencu koraka i razmenu poruka između Korisnika, Forme (korisničkog interfejsa) i Sistema (poslovne logike) prilikom procesa kreiranja novog novčanika.

Detaljan tok procesa:

1. **Inicijalizacija:** Korisnik započinje proces pozivanjem akcije `odaberiOpciju("Dodaj novčanik")` na korisničkom interfejsu (Forma).
2. **Unos podataka:** Korisnik unosi neophodne atribute za novi novčanik, konkretno naziv i valutu, putem poruke `unosPodataka(naziv, valuta)`.
3. **Sistemska obrada:** Forma prosleđuje ove podatke sistemskom kontroleru pozivom metode `kreirajNovcanik(Novcanik)`.
4. **Validacija i alternativni tokovi (Alt fragment):** Sistem vrši proveru validnosti podataka, što rezultira jednim od dva moguća ishoda:
 - **Uspešan ishod ([Podaci validni]):** Ukoliko novčanik sa tim nazivom ne postoji i podaci su ispravni, sistem vraća potvrdu "Novčanik uspešno kreiran". Forma zatim prikazuje korisniku poruku o uspehu.
 - **Neuspešan ishod ([Naziv zauzet]):** Ukoliko novčanik sa unetim nazivom već postoji, sistem vraća grešku "Greška: Novčanik već postoji". Forma obaveštava korisnika o neuspehu.



Slika 6 - Dijagram sekvenci za SK8 - Unos transakcije

Detaljan tok procesa:

1. Inicijalizacija:

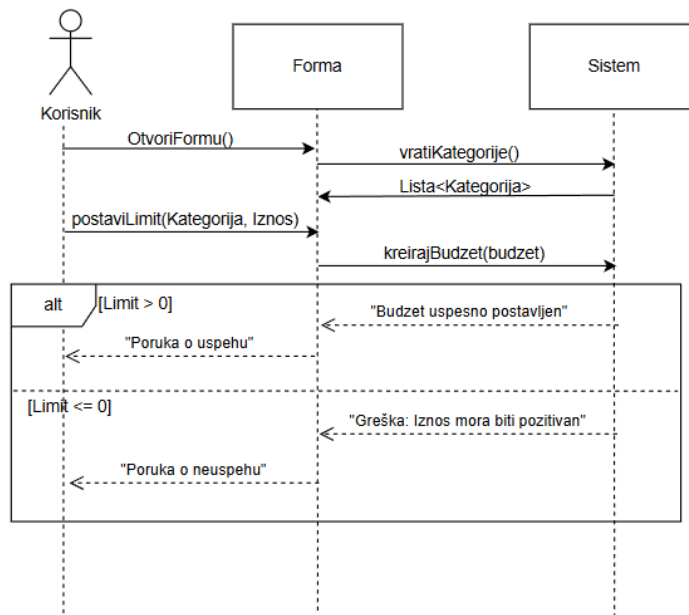
- Proces počinje kada korisnik zatraži otvaranje forme pozivom `OtvoriFormu()`.
- Pre prikaza, Forma automatski šalje zahteve sistemu: `vratiKategorije()` i `vratiNovcanike()`.
- Sistem vraća `Lista<Kategorija>` i `Lista<Novcanik>`, čime se omogućava korisniku da izabere postojeće vrednosti iz padajućih menija, umesto da ih ručno kuca.

2. Unos i slanje podataka:

- Korisnik popunjava formu parametrima: *iznos, tip, kategorija i wallet (novčanik)* putem poruke `unosPodataka(...)`.
- Forma prosleđuje ove podatke sistemu pozivom metode `sacuvajTransakciju(Transakcija)`.

3. Validacija i ishod: Sistem vrši provere (npr. da li je iznos pozitivan i da li su sva polja popunjena).

- **Uspešan ishod:** Ako su `[Podaci validni]`, sistem čuva transakciju i vraća potvrdu "Transakcija uspešno sačuvana". Korisnik dobija poruku o uspehu.
- **Neuspešan ishod:** Ako je zadovoljen uslov `[Iznos <= 0 ili Nedostaje polje]`, sistem detektuje grešku i vraća poruku "Greška: Nevalidni podaci". Korisnik dobija obaveštenje o neuspehu.



Slika 7 - Dijagram sekvenci za SK11 – Postavljanje mesečnog budžeta

Detaljan tok procesa:

1. Inicijalizacija i učitavanje šifarnika:

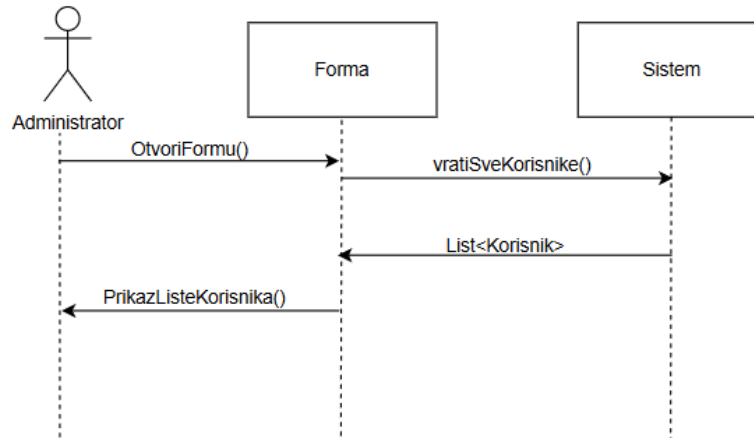
- Korisnik pokreće akciju OtvoriFormu().
- Pre nego što se forma prikaže i omogući unos, ona automatski šalje zahteve sistemu za dobavljanje neophodnih podataka: vratiKategorije() i vratiNovcanike().
- Sistem vraća liste kategorija i novčanika, čime se osigurava da korisnik prilikom unosa bira postojeće vrednosti.

2. Unos podataka i slanje:

- Korisnik popunjava podatke: iznos, tip, kategorija i wallet (novčanik) i šalje ih putem poruke unosPodataka.
- Forma prosleđuje objekat transakcije sistemu pozivom metode sacuvajTransakciju(Transakcija).

3. Validacija: Sistem proverava ispravnost podataka (prikazano kroz alt fragment):

- **Uspešan ishod:** Ako su [Podaci validni], sistem vraća potvrdu "Transakcija uspesno sacuvana", a korisnik dobija poruku o uspehu.
- **Neuspešan ishod:** Ako je ispunjen uslov [Iznos <= 0 ili Nedostaje polje], sistem vraća grešku "Greška: Nevalidni podaci", a korisniku se prikazuje poruka o neuspehu.



Slika 8 - Dijagram sekvenci za SK14 – Pregled svih korisnika

Detaljan tok procesa:

1. Inicijalizacija i priprema podataka:

- Korisnik inicira proces otvaranjem forme za budžet pozivom `OtvoriFormu()`.
- Forma automatski šalje zahtev sistemu `vratiSveKorisnike()` kako bi korisniku ponudila validne opcije.
- Sistem vraća listu dostupnih kategorija (`Lista<Kategorija>`) koje se učitavaju u interfejs.

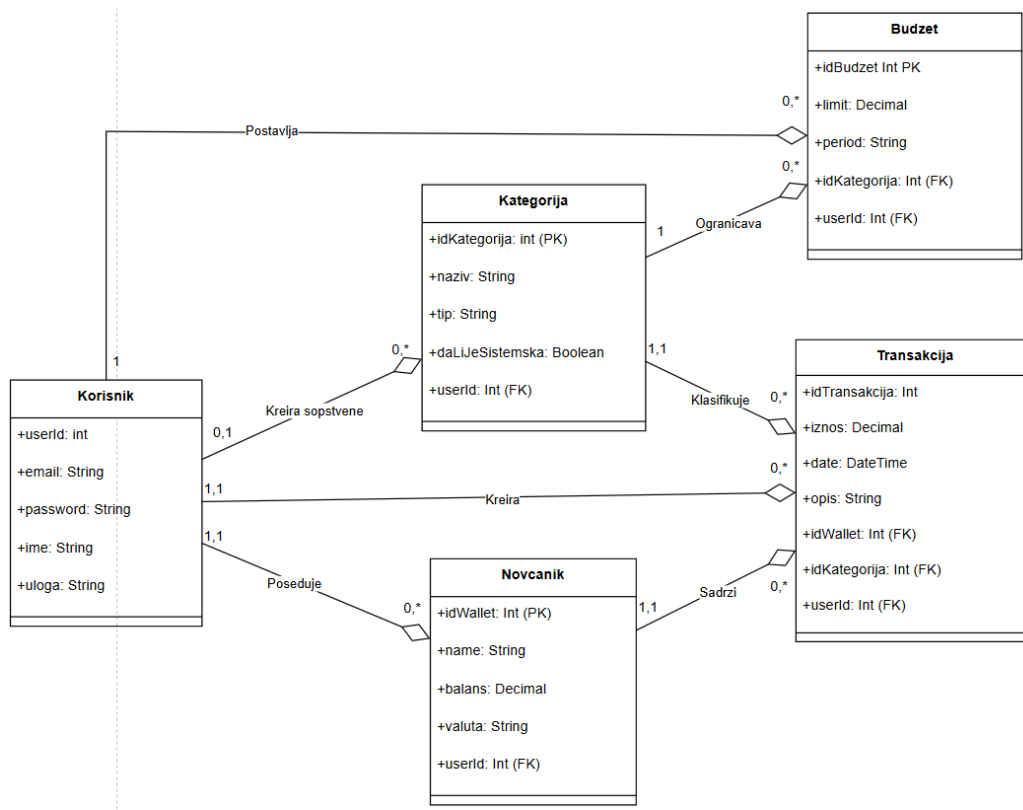
2. Definisanje limita:

- Korisnik bira željenu kategoriju i unosi maksimalni iznos potrošnje pozivom `postaviLimit(Kategorija, Iznos)`.
- Forma šalje zahtev za kreiranje budžeta sistemu metodom `kreirajBudzet(budzet)`.

3. Validacija iznosa: Sistem proverava da li je uneti limit logičan (pozitivan broj):

- **Uspešan ishod:** Ako je ispunjen uslov [`Limit > 0`], sistem uspešno kreira budžet i vraća poruku "Budzet uspesno postavljen", o čemu forma obaveštava korisnika.
- **Neuspešan ishod:** Ako je [`Limit <= 0`], sistem odbija zahtev uz poruku "Greška: Iznos mora biti pozitivan", a korisnik dobija obaveštenje o grešci.

2.4. Model podataka – PMOV ili Dijagram klasa



Slika 9 - Dijagram klasa

Na slici je prikazan logički model baze podataka koji se sastoji od pet međusobno povezanih entiteta: **Korisnik** (User), **Novčanik** (Wallet), **Kategorija** (Category), **Transakcija** (Transaction) i **Budžet** (Budget).

Centralni entitet sistema je **Korisnik**, koji je u relaciji *jedan-prema-više* (1:N) sa svim ostalim entitetima, što znači da svaki podatak u sistemu ima svog vlasnika. **Transakcija** predstavlja asocijativni entitet koji povezuje finansijski izvor (**Novčanik**) sa tipom troška (**Kategorija**). Entitet **Budžet** je direktno vezan za **Kategoriju**, čime se omogućava planiranje i ograničavanje potrošnje po specifičnim tipovima troškova. Integritet podataka osiguran je upotrebom spoljnih ključeva u skladu sa definisanim kardinalnostima.

3. Predlog tehnologija koje će biti korišćene

Za razvoj našeg sistema odabran je moderan tehnološki stek zasnovan na **JavaScript** ekosistemu. Glavni motiv za ovaj izbor je mogućnost korišćenja istog programskog jezika i na klijentskoj (Frontend) i na serverskoj (Backend) strani, što pojednostavljuje razvoj, održavanje i razmenu podataka.

3.1. Frontend

Za razvoj korisničkog interfejsa izabrane su sledeće tehnologije:

- **Next.js (React okvir):** Odabran zbog svoje efikasnosti, modularnosti i ugrađene podrške za rutiranje i serversko renderovanje. React komponentni pristup omogućava kreiranje *reusable* elemenata (poput dugmadi i kartica) koji se koriste kroz celu aplikaciju, čime se ispunjavaju zahtevi za modularnošću.
- **Tailwind CSS:** *Utility-first* CSS okvir koji omogućava brzo i konzistentno stilizovanje aplikacije direktno u HTML/JSX kodu. Obezbeđuje da aplikacija bude responzivna (prilagođena mobilnim i desktop uređajima) bez pisanja kompleksnih CSS fajlova.
- **Axios:** Biblioteka za slanje asinhronih HTTP zahteva ka serveru, koja olakšava komunikaciju sa REST API-jem i rukovanje greškama.

3.2. Backend

Backend deo aplikacije je integrisan u isti projekat putem **Next.js API Routes** funkcionalnosti, a koristi sledeće biblioteke:

- **Node.js (Runtime):** Okruženje koje omogućava izvršavanje JavaScript koda na serveru.
- **Prisma ORM:** Ključna biblioteka za komunikaciju sa bazom podataka. Prisma omogućava definisanje modela podataka u kodu, automatsko generisanje **migracija** (za kreiranje i izmenu tabela) i tipski bezbedne upite ka bazi, čime se ispunjavaju zahtevi za naprednim radom sa podacima.
- **Bcrypt:** Biblioteka za heširanje lozinki, neophodna za ispunjenje bezbednosnih zahteva prilikom registracije i prijave korisnika.
- **JSON Web Token (JWT):** Standard za kreiranje pristupnih tokena koji se koristi za bezbednu autentifikaciju i autorizaciju korisnika na zaštićenim rutama.

3.3. Baza podataka

Za trajno skladištenje podataka izabrana je **Relaciona baza podataka**:

- **MySQL:** Odabrana je zbog svoje pouzdanosti, široke rasprostranjenosti i podrške za relacione modele. S obzirom na prirodu finansijske aplikacije, neophodno je osigurati integritet podataka kroz stroge šeme, primarne i spoljne ključeve (veze između korisnika, novčanika i transakcija), što MySQL u kombinaciji sa Prismom savršeno omogućava.

3.4. Integracije

Sistem je projektovan da komunicira interno i eksterno putem standardnih protokola:

- **REST API Komunikacija:** Osnovni vid integracije između Frontend i Backend dela aplikacije. Aplikacija izlaže sopstvene API krajnje tačke (endpoints) koje vraćaju podatke u JSON formatu.
- **Eksterni servisi:** Arhitektura podržava buduću integraciju sa servisima za slanje transakcionih email-ova (npr. *SendGrid* za potvrdu registracije) i API-jima za konverziju valuta (npr. *Open Exchange Rates*).

3.5. DevOps

Za upravljanje kodom i procesom razvoja koriste se sledeći alati:

- **GitHub:** Koristi se za verzionisanje koda. Projekat je organizovan u javnom repozitorijumu sa jasnom istorijom izmena (commit-ova), gde su oba člana tima kolaboratori.
- **Docker:** Koristi se za izolaciju razvojnog okruženja (containerization). Putem docker-compose konfiguracije omogućeno je jednostavno lokalno pokretanje MySQL baze podataka unutar kontejnera. Ovo osigurava da svi članovi tima rade u identičnom okruženju, bez potrebe za ručnom instalacijom i podešavanjem servera baze na svojim računarima.
- **Vercel / Render (Deployment):** Za postavljanje aplikacije u produkciono okruženje planirano je korišćenje *Vercel* platforme (optimizovane za Next.js) ili *Render* platforme (za MySQL bazu), koje omogućavaju CI/CD (kontinuiranu integraciju i isporuku) povezivanjem sa GitHub repozitorijumom.

4. Prikaz implementacije i korisničkog interfejsa

Ovo poglavlje posvećeno je tehničkoj realizaciji softverskog rešenja **SmartBudget**. Nakon definisanja funkcionalnih zahteva i projektovanja arhitekture sistema, fokus se pomera na konkretnu implementaciju korišćenjem savremenih veb tehnologija.

Sistem je razvijen kao **Single Page Application (SPA)** korišćenjem **Next.js** okvira na frontend-u, dok se za perzistenciju podataka koristi **MySQL** baza podataka u kombinaciji sa **Prisma ORM-om**. Implementacija je vođena principima čiste arhitekture, modularnosti i ponovne upotrebljivosti koda.

U nastavku poglavlja biće prikazani ključni segmenti programskog koda koji demonstriraju logiku aplikacije, izgled korisničkog interfejsa kroz reprezentativne ekrane, kao i matrica koja povezuje inicijalne zahteve sa realizovanim funkcionalnostima.

4.1. Prikaz delova koda sa objašnjenjima

U okviru ovog potpoglavlja izdvojeni su ključni segmenti programskog koda koji ilustruju arhitekturu aplikacije i logiku obrade podataka. Zbog obimnosti kompletnog izvornog koda, prikazani su samo najvažniji delovi koji su ključni za razumevanje funkcionisanja sistema, kao što su definicija modela podataka, API kontroleri za obradu transakcija i frontend komponente za interakciju sa korisnikom.

Kod je organizovan u skladu sa **MVC (Model-View-Controller)** arhitektonskim obrascem, prilagođenim Next.js ekosistemu (App Router), gde su jasno razdvojeni slojevi za pristup podacima, poslovnu logiku i prezentaciju.

```
model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  password String
  name     String
  role     String   @default("USER")
  isBlocked Boolean @default(false)
  createdAt DateTime @default(now())

  wallets      Wallet[]
  categories    Category[]
  transactions  Transaction[]
  budgets       Budget[]
}
```

Slika 10 - Definicija entiteta Korisnik (Prisma Schema)

Prikazani segment koda (Slika 10) definiše centralni entitet sistema – User (Korisnik). Pored standardnih atributa za identifikaciju (id) i autentifikaciju (email, password), model sadrži i ključna polja za kontrolu pristupa:

- **role:** Definirano kao String sa podrazumevanom vrednošću "USER". Ovo polje omogućava razlikovanje standardnih korisnika od administratora ("ADMIN") ili gostiju ("GUEST"), čime se regulišu prava pristupa funkcionalnostima aplikacije.
- **isBlocked:** Logičko polje (Boolean) koje služi kao sigurnosni mehanizam, omogućavajući administratorima da po potrebi suspenduju pristup nalogu.

Takođe, definisane su relacije prema svim ostalim entitetima u sistemu (wallets, transactions, budgets), što omogućava da se svi finansijski podaci jednoznačno vežu za konkretnog korisnika.

```
model Transaction {
  id          Int          @id @default(autoincrement())
  amount      Decimal
  type        String
  date        DateTime @default(now())
  description String?

  walletId Int
  wallet    Wallet @relation(fields: [walletId], references: [id], onDelete: Cascade)

  categoryId Int
  category    Category @relation(fields: [categoryId], references: [id])

  userId Int
  user      User @relation(fields: [userId], references: [id])
}
```

Slika 11 - Definicija entiteta Transakcija i relacija

Model **Transaction** predstavlja srž finansijske logike aplikacije. Ovaj entitet povezuje novčane tokove sa korisnicima, novčanicima i kategorijama.

- **Tip transakcije (type):** Čuva se kao String i koristi se za razlikovanje tri osnovna tipa finansijskih aktivnosti: prihoda ("**INCOME**"), rashoda ("**EXPENSE**") i internih prenosa ("**TRANSFER**").
- **Finansijska preciznost:** Za čuvanje novčanih iznosa (amount) koristi se tip Decimal umesto standardnog Float-a, čime se izbegavaju greške u zaokruživanju karakteristične za rad sa novcem u programiranju.
- **Relacije i Integritet:** Korišćenje direktive onDelete: Cascade na relaciji sa novčanikom (wallet) osigurava referencijalni integritet baze. To znači da ukoliko korisnik odluči da obriše određeni novčanik, baza podataka će automatski i bezbedno ukloniti sve transakcije koje su bile vezane za taj novčanik, sprečavajući nastanak nekonzistentnih podataka.


```

const transaction = await tx.transaction.create({
  data: {
    amount: parseFloat(amount),
    type,
    date: new Date(date),
    description,
    walletId: parseInt(walletId),
    categoryId: parseInt(categoryId),
    userId: wallet.userId
  },
});

if (type === "INCOME") {
  await tx.wallet.update({
    where: { id: parseInt(walletId) },
    data: { balance: { increment: parseFloat(amount) } },
  });
} else {
  await tx.wallet.update({
    where: { id: parseInt(walletId) },
    data: { balance: { decrement: parseFloat(amount) } },
  });
}

```

Slika 12 - Implementacija atomske transakcije za očuvanje integriteta finansijskih podataka

Prikazani isečak koda demonstrira implementaciju **ACID** transakcija na backend-u korišćenjem Prisma ORM-a. Ovo je ključni deo sistema za očuvanje integriteta finansijskih podataka.

Korišćenjem metode **prisma.\$transaction**, obezbeđeno je atomsko izvršavanje operacija:

1. Kreiranje novog zapisa u tabeli **Transaction**.
2. Ažuriranje stanja (balance) u povezanoj tabeli **Wallet**.

Logika je implementirana tako da se stanje novčanika automatski uvećava ili umanjuje u zavisnosti od tipa transakcije (**INCOME** ili **EXPENSE**). Ukoliko bilo koji korak u ovom procesu ne uspe (npr. greška pri upisu u bazu), cela operacija se poništava (rollback), čime se sprečava situacija da transakcija bude zabeležena, a stanje novčanika nepromenjeno.

```

const filteredTransactions = transactions.filter((tx) => {
  const matchesSearch = tx.description?.toLowerCase().includes(searchTerm.toLowerCase()) ||
    tx.wallet?.name.toLowerCase().includes(searchTerm.toLowerCase());
  const matchesType = filterType === "ALL" || tx.type === filterType;
  const categoryName = tx.category?.name || "Ostalo";
  const matchesCategory = filterCategory === "ALL" || categoryName === filterCategory;

  return matchesSearch && matchesType && matchesCategory;
});

```

Slika 13 - Logika za dinamičko filtriranje podataka

Slika 13 prikazuje implementaciju naprednog mehanizma za pretragu i filtriranje podataka na strani klijenta (**Client-side filtering**). Umesto slanja novog zahteva bazi za svaku promenu filtera, aplikacija koristi reaktivnost React biblioteke za trenutno ažuriranje prikaza.

Algoritam koristi metodu **Array.prototype.filter()** koja kombinuje tri nezavisna kriterijuma logičkim AND operatorom:

1. **Tekstualna pretraga:** Proverava se da li opis transakcije ili naziv novčanika sadrže uneti termin (case-insensitive).
2. **Filter tipa:** Selektovanje samo prihoda, troškova ili transfera.
3. **Filter kategorije:** Filtriranje po specifičnoj kategoriji troška.

Ovakav pristup značajno poboljšava korisničko iskustvo (User Experience) jer je odziv interfejsa trenutno, a smanjuje se i opterećenje servera.

```
// proveravamo da li je korisnik blokiran
if (user.isBlocked) {
  return NextResponse.json(
    { message: 'Vaš nalog je blokiran. Kontaktirajte administratora.' },
    { status: 403 } // 403 znaci zabranjeno
  );
}

const passwordMatch = await bcrypt.compare(password, user.password);
```

Slika 14 - Implementacija autentifikacije sa hešovanjem lozinke i sigurnosnim proverama

Ovaj segment koda ilustruje proces autentifikacije korisnika na API ruti **/api/login**.

Implementirane su višeslojne bezbednosne provere pre izdavanja sesije korisniku.

Prvo, sistem proverava status naloga proverom polja **isBlocked**. Ako je administrator blokirao korisnika, server odbija prijavu i vraća status **403 Forbidden**, čime se efikasno zabranjuje pristup nepoželjnim korisnicima bez brisanja njihovih podataka.

Drugo, koristi se biblioteka **bcrypt** za bezbedno poređenje unete lozinke sa hešovanom (salted & hashed) lozinkom koja je sačuvana u bazi podataka. Ovim se osigurava da se lozinke nikada ne čuvaju niti prenose u čitljivom formatu, što je standard u zaštiti korisničkih podataka.

```
import { PrismaClient } from '@prisma/client';

const prismaClientSingleton = () => {
  return new PrismaClient();
};

type PrismaClientSingleton = ReturnType<typeof prismaClientSingleton>;

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClientSingleton | undefined;
};

const prisma = globalForPrisma.prisma ?? prismaClientSingleton();

export default prisma;

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Slika 15 - Implementacija Prisma klijenta kao Singleton servisa

Prikazan je infrastrukturni servis (prisma.ts) koji je odgovoran za uspostavljanje i održavanje konekcije sa bazom podataka.

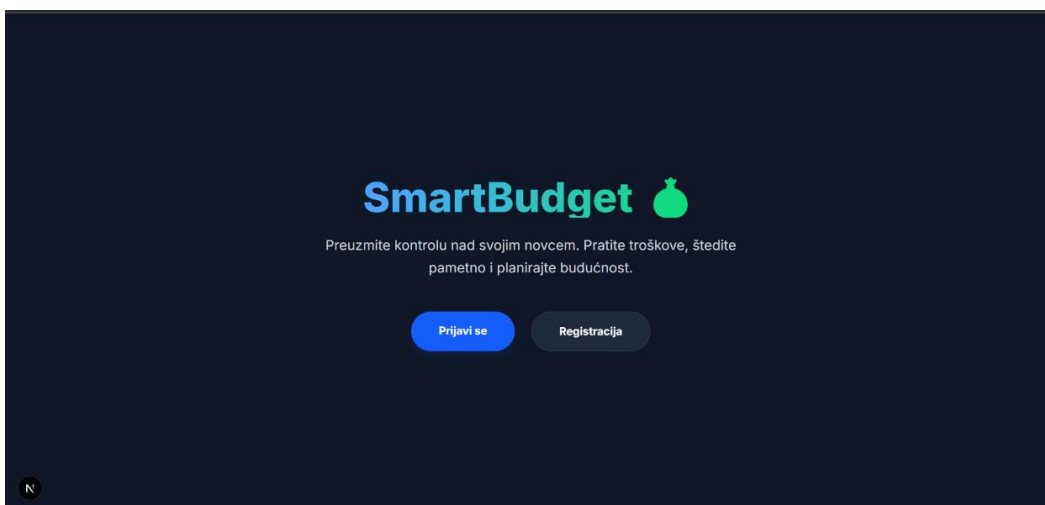
Implementiran je **Singleton dizajn patern** koji osigurava da postoji samo jedna aktivna instanca Prisma klijenta u celoj aplikaciji. Ovo je ključno za Next.js okruženje jer sprečava iscrpljivanje konekcija ka bazi podataka (Connection Pool Exhaustion) prilikom 'hot-reload' funkcionalnosti tokom razvoja softvera.

4.2. Prikaz korisničkog interfejsa (Frontend UI)

U ovom poglavlju prezentovan je vizuelni identitet aplikacije SmartBudget, koji predstavlja direktnu manifestaciju prethodno opisane programske logike. Korisnički interfejs (UI) je dizajniran sa fokusom na intuitivnost, preglednost i efikasnost, omogućavajući korisnicima da brzo i jednostavno upravljaju svojim finansijama.

Za stilizovanje komponenata korišćen je Tailwind CSS okvir, što je omogućilo kreiranje modernog dizajna koji se automatski prilagođava različitim veličinama ekrana.

U nastavku su prikazani reprezentativni ekrani aplikacije, praćeni opisom dostupnih funkcionalnosti i interakcija koje korisnik može ostvariti na svakom od njih.

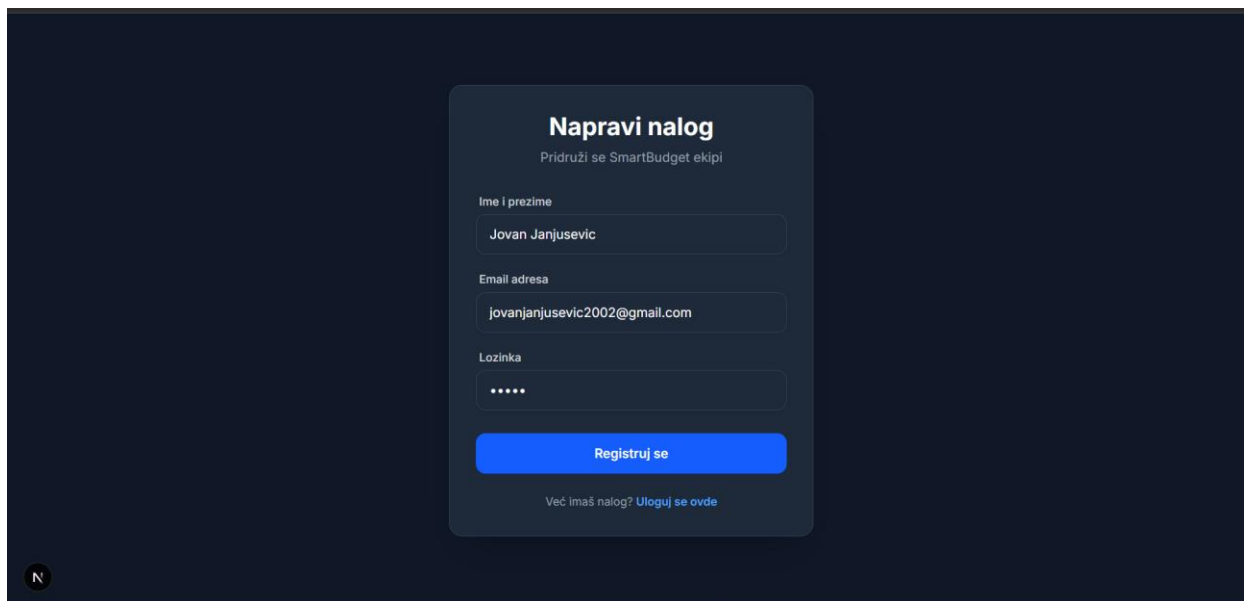


Slika 16 - Početna stranica za gostujućeg korisnika (Guest View)

Prikazani ekran ilustruje implementaciju sistema privilegija i zaštite podataka. Kada se na sistem prijavi korisnik sa dodeljenom ulogom "**GUEST**" (Gost), aplikacija ga automatski preusmerava na poseban prikaz, izolovan od osetljivih finansijskih podataka.

Funkcionalnosti i ograničenja:

- **Ograničen pristup:** Za razliku od standardnih korisnika, gost nema pristup modulima za upravljanje novčanicima, transakcijama i budžetima. Pokušaj pristupa ovim rutama biće blokiran na nivou *Middleware*-a ili *Frontend* logike.
- **Prilagođeni interfejs:** Navigacioni meni (Navbar) i podnožje (Footer) se automatski prilagođavaju i sakrivaju linkove ka zaštićenim delovima aplikacije, ostavljajući dostupnom samo opciju za odjavu sa sistema.
- **Informativni sadržaj:** Korisniku se prikazuje jasna poruka o statusu naloga i instrukcije o potrebi za registracijom ukoliko želi pun pristup funkcionalnostima.

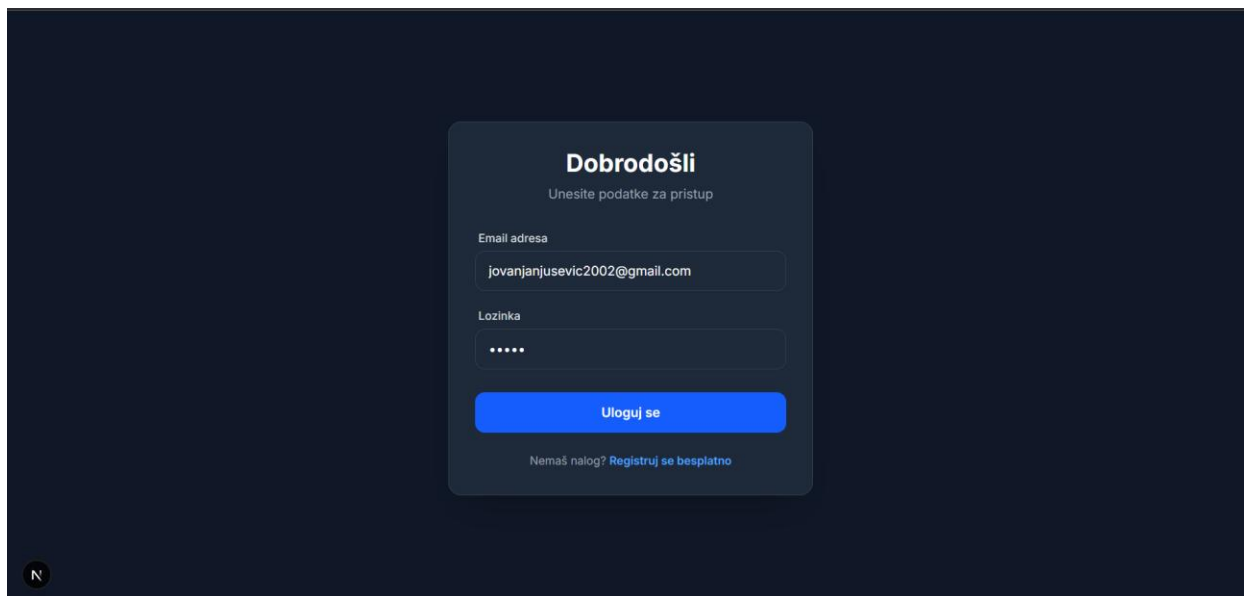


Slika 17 - Forma za registraciju novih korisnika

Ekran za registraciju predstavlja ulaznu tačku u sistem za nove korisnike. Dizajniran je minimalistički kako bi fokus bio na samom procesu kreiranja naloga, bez nepotrebnih distrakcija.

Ključne funkcionalnosti:

- **Prikupljanje podataka:** Forma zahteva unos osnovnih identifikacionih podataka neophodnih za kreiranje profila: **Ime i prezime** (za personalizaciju interfejsa), **Email adresa** (kao jedinstveni identifikator) i **Lozinka**.
- **Validacija unosa:** Implementirana je validacija na klijentskoj strani koja osigurava da su sva polja popunjena i da je email u ispravnom formatu pre nego što se podaci pošalju na server. Ovo smanjuje nepotrebne zahteve ka API-ju i poboljšava odziv aplikacije.
- **Povratna informacija:** Korisnik dobija jasne poruke u slučaju greške (npr. "Email je već u upotrebi") ili uspešne registracije.
- **Navigacija:** Postoji jasan link ka stranici za prijavu ("*Već imate nalog? Prijavi se*"), čime se olakšava navigacija korisnicima koji su greškom došli na ovaj ekran.



Slika 18 - Ekran za prijavu na sistem (Login)

Ekran za prijavu dizajniran je u minimalističkom stilu sa fokusom na jednostavnost korišćenja. U centralnom delu ekrana nalazi se **pristupna forma** koja od korisnika zahteva unos dva podatka: Email adrese i Lozinke.

Elementi interfejsa i interakcija:

- **Polja za unos:** Jasno definisana polja sa *placeholder* tekstom koji korisniku sugeriše šta treba da upiše.
- **Akciono dugme:** Dugme "Prijavi se" je vizuelno istaknuto plavom bojom kako bi privuklo pažnju i jasno ukazalo na sledeći korak. Klikom na ovo dugme pokreće se provera podataka.
- **Povratne informacije:** U slučaju pogrešno unetih podataka, korisniku se na istom ekranu prikazuje crvena poruka upozorenja, bez potrebe za osvežavanjem stranice.
- **Navigacija ka registraciji:** Ispod forme nalazi se link "*Nemaš nalog? Registruj se*", koji omogućava novim korisnicima lak prelazak na ekran za kreiranje naloga.



Slika 19 - Glavna kontrolna tabla korisnika (User Dashboard)

Nakon uspešne prijave, korisnik se usmerava na Dashboard, koji pruža sveobuhvatan pregled finansijskog stanja u realnom vremenu. Interfejs je podeljen u tri ključne sekcije radi lakše preglednosti:

1. Statistika i pregled stanja (Leva kolona):

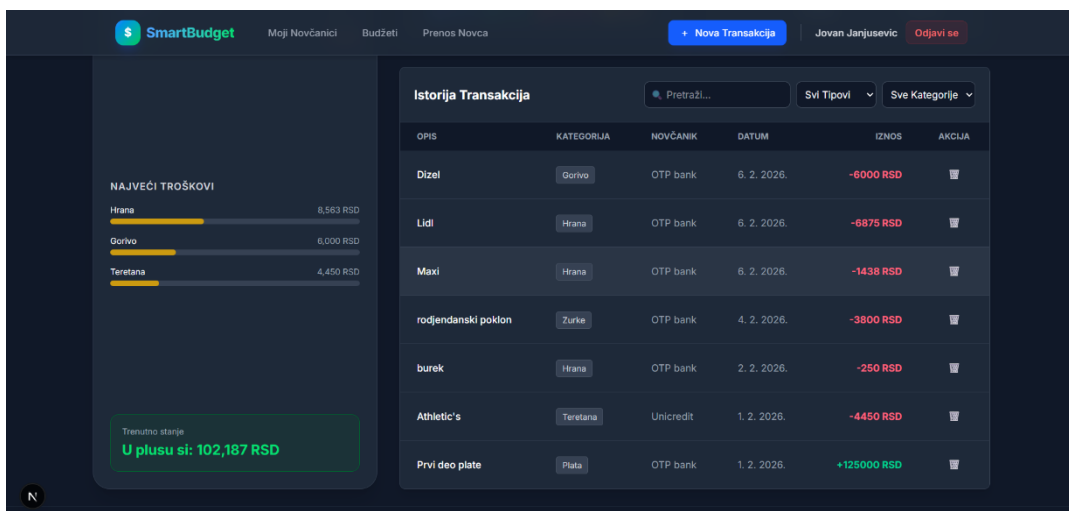
- **Personalizovani pozdrav:** Na vrhu ekrana prikazuje se ime korisnika, pružajući osećaj personalizacije.
- **Kartice sa ključnim indikatorima:** Prikazana su tri ključna finansijska parametra: **Trenutni saldo:** Ukupan raspoloživ novac na svim novčanicima, **ukupni prihodi:** zbir svih priliva u tekućem mesecu (označeno zelenom bojom) i **ukupni rashodi:** zbir svih troškova u tekućem mesecu (označeno crvenom bojom).

2. Vizuelizacija podataka (Gornji desni deo):

- **Struktura troškova (Pie Chart):** Kružni dijagram koji korisniku vizuelno prikazuje na šta najviše troši novac (npr. Hrana, Stanarina, Zabava). Svaki segment je jasno obojen, a prelazaskom miša prikazuje se tačan iznos.
- **Odnos Prihoda i Rashoda (Bar Chart):** Stubasti dijagram koji omogućava brzo poređenje koliko je novca zarađeno u odnosu na to koliko je potrošeno.

3. Eksterni servisi i korisne informacije:

- **Aktuelna kursna lista:** Zahvaljujući integraciji sa eksternim API servisom, korisnik može pratiti kretanje kursa evra u odnosu na dinar u realnom vremenu. Ova funkcionalnost je od velikog značaja za planiranje troškova ili štednje u stranim valutama.
- **Finansijski savet dana:** Putem AdviceSlip servisa, na interfejsu se generiše nasumična poruka dana. Ovi kratki saveti služe kao dodatna motivacija korisniku ili pružaju korisne sugestije za opšte upravljanje životnim navikama i finansijama.



Slika 20 - Tabela istorije transakcija sa naprednim filterima

Donji segment kontrolne table rezervisan je za detaljan tabelarni prikaz finansijskih aktivnosti. Ova komponenta ne služi samo za pasivan pregled, već pruža alate za aktivno upravljanje podacima.

Elementi i interakcije:

- **Traka sa alatima (Toolbar):** Iznad same tabele nalazi se set kontrola za filtriranje podataka u realnom vremenu:
 - **Polje za pretragu:** Omogućava brzo pronalaženje transakcija kucanjem ključnih reči (npr. "Maxi", "Plata").
 - **Filter po tipu:** Padajući meni za prikazivanje samo prihoda, rashoda ili transfera.
 - **Filter po kategoriji:** Omogućava izolovanje specifičnih grupa troškova (npr. samo "Hrana" ili "Prevoz").
 - **Dugme za poništavanje:** Pojavljuje se kada su filteri aktivni, omogućavajući brz povratak na kompletan pregled.
- **Tabelarni prikaz:** Podaci su organizovani u kolonama radi lakše čitljivosti: Opis, Kategorija (sa vizuelnom oznakom), Novčanik sa kog je plaćeno, Datum i Iznos. Iznosi su obojeni različito (zeleno za prihode, crveno za rashode) radi bržeg vizuelnog skeniranja.
- **Akcije nad podacima:** Poslednja kolona "Akcija" sadrži kontrole za upravljanje pojedinačnim zapisima. Implementirana je funkcija **brisanja transakcije** (ikonica kante), koja automatski ažurira i stanje na povezanom novčaniku, čime se održava konzistentnost podataka.

Slika 21- Forma za evidentiranje nove transakcije

Ekran „Nova transakcija” omogućava korisniku da unese novu finansijsku transakciju u sistem, bilo da je u pitanju trošak ili prihod. Ovaj ekran predstavlja centralnu tačku za evidentiranje svih finansijskih aktivnosti korisnika u aplikaciji SmartBudget.

Na vrhu ekrana nalazi se navigaciona traka sa nazivom aplikacije *SmartBudget*, linkovima ka glavnim sekcijama aplikacije (Moji novčanici, Budžeti, Prenos novca), kao i dugmetom za dodavanje nove transakcije i opcijom za odjavu korisnika.

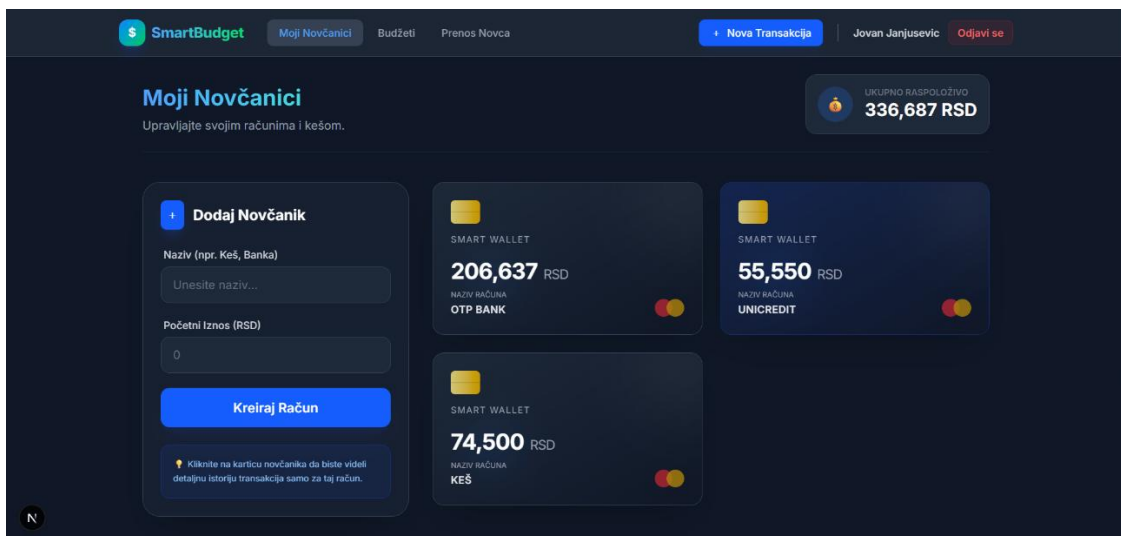
Centralni deo ekrana sadrži formu za unos transakcije. Na početku forme korisnik bira tip transakcije putem prekidača:

- **Trošak**
- **Prihod**

U zavisnosti od izabranog tipa, transakcija se kasnije različito obrađuje u sistemu (umanjenje ili uvećanje stanja novčanika). Forma za unos sadrži sledeća polja:

- **Iznos** – numeričko polje u koje korisnik unosi vrednost transakcije.
- **Opis** – tekstualno polje za kratak opis transakcije (npr. naziv prodavnice ili svrha troška).
- **Novčanik** – padajuća lista iz koje se bira novčanik na koji se transakcija odnosi (npr. keš, kartica).
- **Kategorija** – padajuća lista kategorija troškova ili prihoda (npr. hrana, plata).
- **Datum** – polje za izbor datuma transakcije putem kalendara.

Na dnu forme nalazi se primarno dugme „**Sačuvaj transakciju**”, koje potvrđuje unos i trajno čuva podatke u sistemu, kao i opcija „**Odustani**” koja omogućava povratak bez čuvanja podataka.



Slika 22 - Upravljanje novčanicima (Moji Novčanici)

Ovaj ekran predstavlja centralni deo aplikacije za upravljanje finansijskim izvorima. Omogućava korisniku pregled ukupnog stanja, kao i realizaciju slučajeva korišćenja **SK4 (Kreiranje novčanika)** i **SK6 (Brisanje novčanika)**. Korisnički interfejs je podeljen u nekoliko logičkih celine:

Zaglavlje sa zbirnim stanjem:

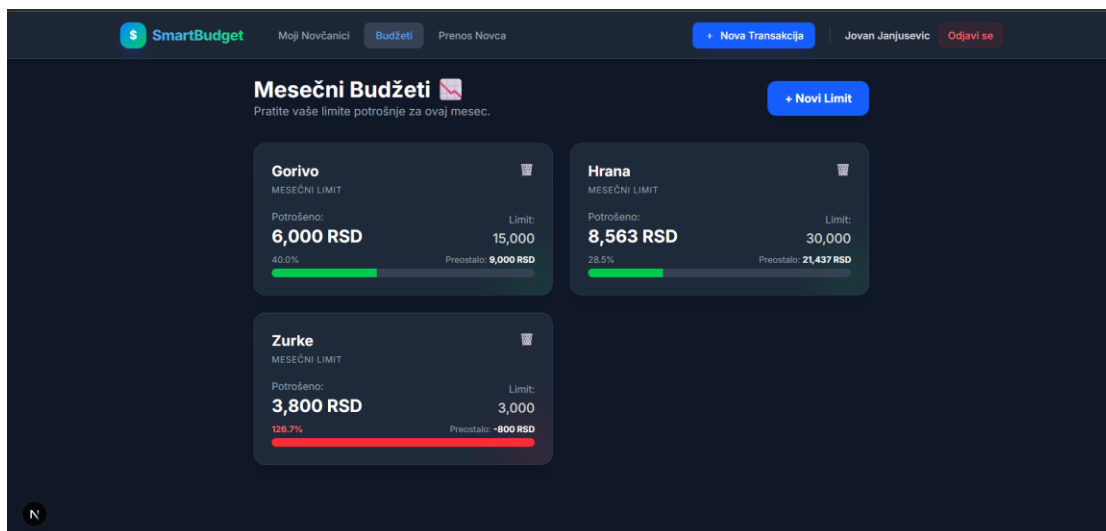
- Na vrhu stranice nalazi se istaknuta sekcija koja prikazuje "**Ukupno raspoloživo**" stanje. Ova vrednost se automatski izračunava sumiranjem sredstava sa svih korisnikovih novčanika, pružajući trenutni uvid u finansijsku situaciju.

Forma za dodavanje novčanika (Leva strana):

- Ovaj panel služi za unos novih izvora sredstava (**SK4**).
- Sadrži polja za unos: **Naziv:** (npr. "Keš", "Banka", "Štednja") i **početni iznos:** Numeričko polje za definisanje startnog balansa.
- Klikom na dugme "**Kreiraj Račun**", podaci se validiraju i šalju na server, a novi novčanik se momentalno pojavljuje na listi desno.

Lista aktivnih novčanika (Desna strana):

- Prikazuje sve korisnikove novčanike u formi stilizovanih kartica koje podsećaju na kreditne kartice.
- Svaka kartica prikazuje naziv novčanika i trenutno stanje u RSD.
- **Interakcije na kartici: Pregled istorije:** Klikom na samu karticu otvara se **modalni prozor** koji prikazuje detaljnu tabelu transakcija (istoriju priliva i odliva) vezanih isključivo za taj novčanik. **Brisanje (SK6):** Prelaskom miša preko kartice (ili dodirrom na mobilnom uređaju) pojavljuje se ikonica za brisanje. Klikom na nju aktivira se sigurnosni mehanizam – iskaćući prozor (modal) koji traži od korisnika da potvrdi trajnu akciju brisanja novčanika i svih njegovih transakcija.



Slika 23 -Mesečni Budžeti (Limiti potrošnje)

Ovaj ekran omogućava korisniku da definiše i prati svoje mesečne limite potrošnje po kategorijama ili za celokupan budžet. Glavni cilj je vizuelizacija stepena potrošnje u realnom vremenu. Naslovna sekcija: U gornjem delu se nalazi naslov "Mesečni Budžeti" sa kratkim obaveštenjem, kao i istaknuto plavo dugme "+ Novi Limit" koje pokreće proces planiranja. Kartice budžeta (Grid): Centralni deo ekrana zauzimaju kartice. Svaka kartica predstavlja jedan postavljeni limit i sadrži:

- Naziv kategorije: (npr. "Hrana" ili "Ukupan budžet").
- Ikonica kante: Diskretna ikona za brisanje limita u gornjem desnom uglu kartice.
- Statistika potrošnje: Prikaz tačnog iznosa koji je do sada potrošen u odnosu na postavljeni limit (npr. "15,000 RSD / 20,000 RSD").
- Progress Bar: Dinamička linija napretka koja menja boju u zavisnosti od popunjenosti:
 - Zelena: Potrošnja je ispod 50% limita.
 - Žuta: Potrošnja je između 50% i 85%.
 - Crvena: Potrošnja je prešla 85% ili je limit prekoračen.
- Preostala sredstva: Jasno istaknut preostali iznos do kraja meseca.
- Prozor za kreiranje: Sadrži padajući meni za izbor kategorije i polje za unos maksimalnog iznosa.
- Prozor za potvrdu: Sigurnosni upit koji se pojavljuje pre brisanja limita.
- Postavljanje limita: Klikom na dugme za novi limit, korisnik otvara formu gde može da izabere kategoriju troška (iz baze podataka) i dodeli joj novčani limit.
- Monitoring u realnom vremenu: Korisnik prati procenite i boje na karticama kako bi odmah uvideo gde najviše troši i koliko mu je novca ostalo.
- Upravljanje (Brisanje): Ukoliko neki limit više nije potreban, korisnik ga može obrisati uz prethodnu potvrdu, čime se kartica uklanja sa ekrana i iz baze podataka.

Slika 24 -Interni prenos sredstava

Ekran „Interni prenos” omogućava korisniku da prebaci sredstva između sopstvenih novčanika u okviru aplikacije SmartBudget. Ova funkcionalnost služi za interno upravljanje finansijama, bez evidentiranja prihoda ili rashoda, već isključivo preraspodelu postojećih sredstava.

Prikaz korisničkog interfejsa:

Ekran je organizovan kao centralizovana forma smeštena u sredini stranice, sa jasno istaknutim naslovom *Interni prenos*. Dizajn je u tamnom režimu, u skladu sa ostatkom aplikacije, uz fokus na preglednost i jednostavnost korišćenja.

Forma za prenos sredstava sadrži sledeće elemente:

- **Izvorni novčanik (Sa novčanika – šalje)** – padajuća lista iz koje korisnik bira novčanik sa kog se sredstva prenose. Pored naziva novčanika prikazuje se trenutno stanje i valuta.
- **Ciljni novčanik (Na novčanik – prima)** – padajuća lista za izbor novčanika na koji se sredstva prenose. Izvorni novčanik je automatski onemogućen za izbor kako bi se sprečile greške.
- **Iznos za prenos** – numeričko polje za unos iznosa koji se prenosi.
- **Opis (opciono)** – tekstualno polje koje omogućava korisniku da navede svrhu prenosa (npr. štednja, prebacivanje sredstava).
- **Datum** – polje za izbor datuma izvršenja prenosa.

Na dnu forme nalazi se primarno dugme „**Izvrši prenos**”, koje potvrđuje operaciju, kao i sekundarna opcija „**Odustani**” za povratak na prethodnu stranicu bez izvršavanja prenosa. U slučaju greške (npr. izbor istog izvornog i ciljnog novčanika), korisniku se prikazuje jasno istaknuta poruka o grešci.

Ako korisnik nema minimum dva novčanika, prikazuje se informativna poruka sa pozivom da najpre kreira novi novčanik.

Administratorski panel omogućava administratoru aplikacije SmartBudget centralizovano upravljanje korisnicima i sistemskim kategorijama. Ovaj ekran je dostupan isključivo korisnicima sa administratorskom ulogom i predstavlja kontrolnu tačku za nadzor i održavanje aplikacije.

Prikaz korisničkog interfejsa:

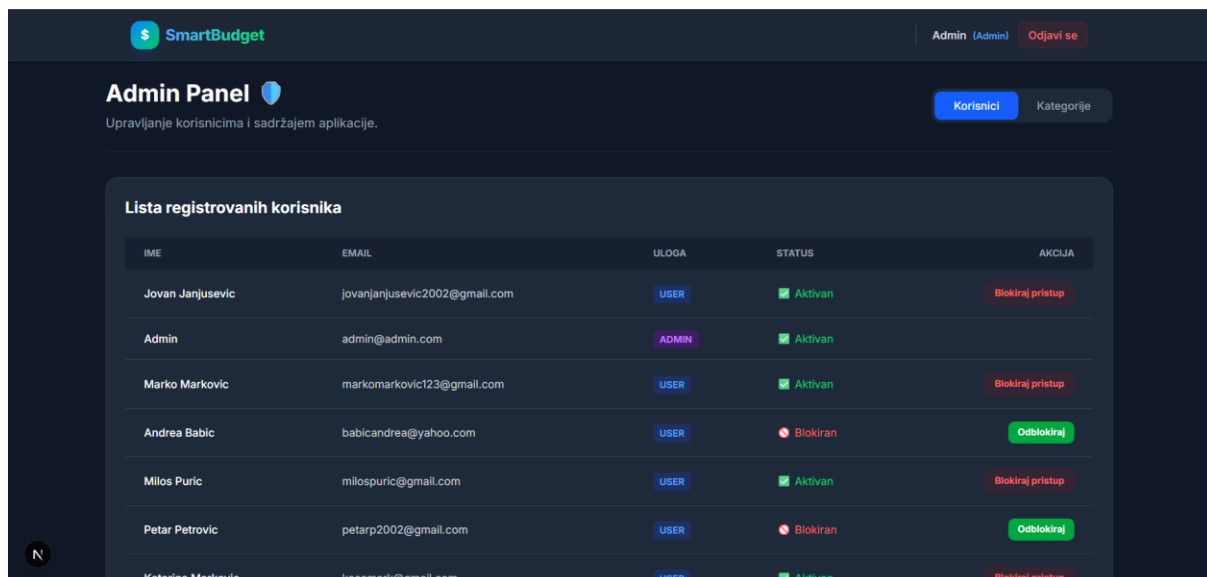
Administratorski panel je prikazan kao zasebna stranica sa jasno istaknutim naslovom *Admin Panel* i kratkim opisom njegove namene. Interfejs je u tamnom režimu, u skladu sa ostatkom aplikacije, sa naglaskom na preglednost i strukturu podataka.

Na vrhu stranice nalazi se sekcija sa:

- naslovom panela,
- kratkim opisom funkcionalnosti,
- tab navigacijom koja omogućava prebacivanje između različitih administratorskih funkcija.

Administratorske funkcionalnosti su podeljene u dva taba:

- **Korisnici**
- **Kategorije**



Slika 25 - Admin Dashboard - Tab Korisnici

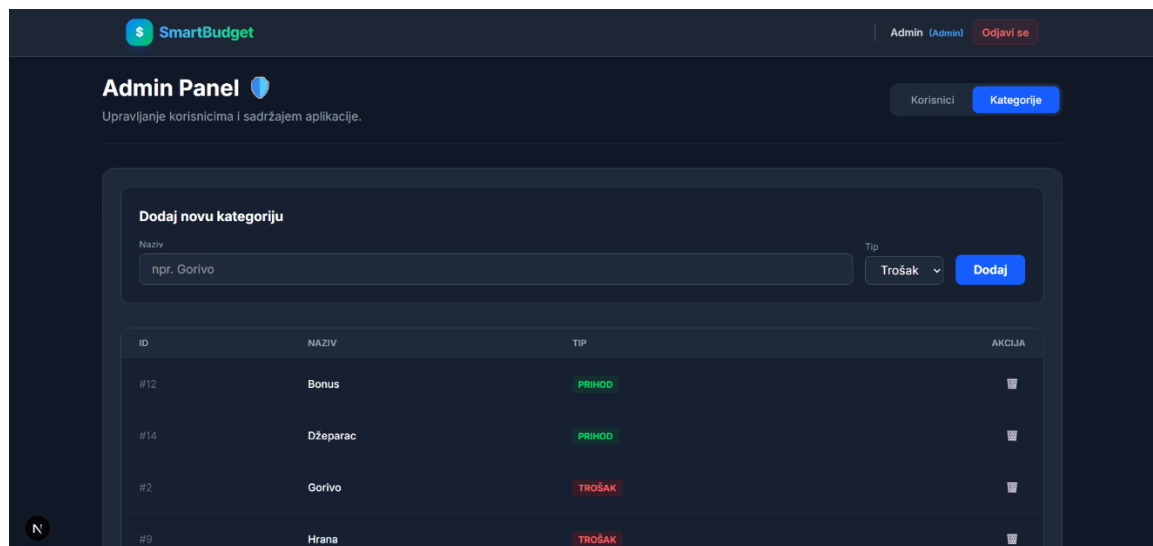
Tab Korisnici

U okviru taba *Korisnici* prikazana je tabela sa listom svih registrovanih korisnika aplikacije.

Tabela sadrži sledeće kolone:

- **Ime korisnika**
- **Email adresa**
- **Uloga** (administrator ili standardni korisnik)
- **Status naloga** (aktivan ili blokiran)
- **Akcija** (blokiranje ili odblokiranje naloga)

Status korisnika je vizuelno naglašen bojama i ikonama radi brze orijentacije. Administrator ima mogućnost da blokira ili odblokira nalog korisnika, osim u slučaju administratorskih naloga, koji su zaštićeni od ovakvih akcija.



Slika 26 - Admin Dashboard - Tab Kategorije

Tab Kategorije

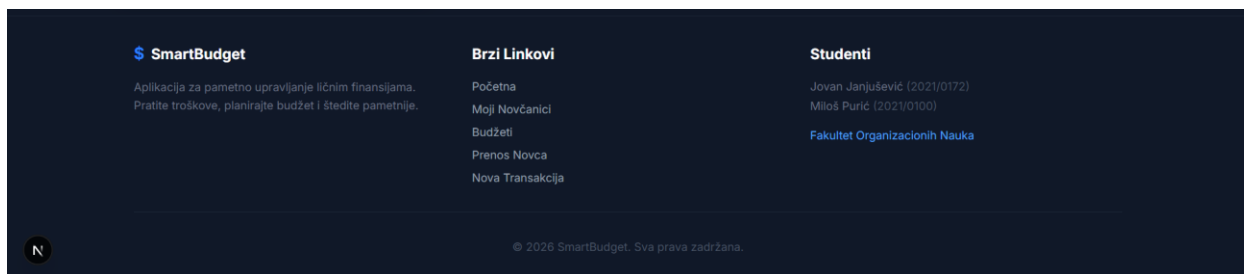
U okviru taba *Kategorije* prikazuje se posebna administratorska komponenta namenjena upravljanju kategorijama koje se koriste u aplikaciji (npr. kategorije troškova i prihoda). Ovaj deo omogućava održavanje sistemskih podataka koji su dostupni svim korisnicima aplikacije. Admin može dodavati nove kategorije, bilo da su to prihodi ili troškovi, a može i klikom na ikonicu kante briste kategorije. Nakon toga mu se otvara modalni prozor u kome mora potvrditi da želi da obriše datu kategoriju.

Administrator se kreće između tabova pomoću tab navigacije. U tabu za korisnike može pregledati sve naloge i menjati njihov status (aktivan/blokiran). Promene se odmah primenjuju u sistemu bez potrebe za ponovnim učitavanjem stranice. U tabu za kategorije administrator upravlja kategorijama koje se kasnije koriste prilikom unosa transakcija.



Slika 27 - Navbar

Navbar – lepljiva (sticky) tamna traka na vrhu stranice sa malim gradient znakom „\$” i nazivom **SmartBudget**. Sadrži glavne navigacione linkove – **Početna**, **Moji Novčanici**, **Budžeti**, **Prenos Novca** (vidljivi na većim ekranima), izdvojeno dugme **Nova Transakcija**, prikaz imena korisnika sa oznakom **(Admin)** kada je uloga admin, i dugme **Odjavi se** koje otvara modal za potvrdu odjave. Navbar se skriva na stranicama za prijavu/registraciju i na početnoj stranici za goste.



Slika 28 - Footer

Tamni donji deo stranice podeljen u tri kolone:

- kratak opis projekta uz logo i naziv **SmartBudget**,
- lista brzih linkova (Početna, Moji Novčanici, Budžeti, Prenos Novca, Nova Transakcija) koja reflektuje navigaciju
- odeljak „Studenti” sa imenima autora i podacima fakulteta. Ispod kolona nalazi se copyright linija sa tekućom godinom. Footer se ne prikazuje na stranicama za prijavu/registraciju i na landing stranicama za goste.

4.3. Povezanost zahteva i implementacije

U narednoj tabeli prikazana je veza između definisanih funkcionalnih zahteva (Slučajeva Korišćenja - SK), softverskih modula (Frontend komponenti), realizovanih API ruta (Backend kontrolera) i korisničkog interfejsa na kojem se funkcionalnost izvršava.

Zahtev	Modul / Komponenta (Kod)	API Ruta (Backend)	UI Ekran (Ruta aplikacije)
SK1 – Registracija korisnika	src/app/register/page.tsx	POST /api/register	/register
SK2 – Prijava korisnika (Login)	src/app/login/page.tsx	POST /api/login	/login
SK3 – Odjava sa sistema	src/components/Navbar.tsx	POST /api/logout	Modalni prozor na svim stranama
SK4 – Kreiranje novčanika	src/app/wallets/page.tsx	POST /api/wallets	/wallets
SK7 – Prenos sredstava	src/app/transfer/page.tsx	POST /api/transfer	/transfer
SK8 – Unos transakcije	src/app/transactions/add/page.tsx	POST /api/transactions	/transactions/add
SK9 – Filtriranje transakcija	src/components/TransactionHistory.tsx	GET /api/transactions	/ (Dashboard)
SK10 – Brisanje transakcije	src/components/TransactionHistory.tsx	DELETE /api/transactions/[id]	/ (Dashboard)
Upravljanje kategorijama (Admin)	src/components/AdminCategories.tsx	POST /api/admin/categories	/ (Admin Dashboard Tab)
Blokiranje korisnika (Admin)	src/components/AdminDashboard.tsx	PUT /api/admin/users/block	/ (Admin Dashboard Tab)

Tabela 3 - Prikaz implementacije funkcionalnih zahteva po komponentama

Realizacija nefunkcionalnih zahteva

Pored funkcionalnih, implementirani su i ključni nefunkcionalni zahtevi kroz sledeće mehanizme:

1. Bezbednost podataka:

- Lozinke se ne čuvaju u otvorenom tekstu, već se hešuju korišćenjem **Bcrypt** algoritma pre upisa u bazu (implementirano u api/register).
- Pristup administrativnim funkcijama zaštićen je proverom uloge korisnika (`role === 'ADMIN'`) na API nivou.

2. Integritet podataka (Pouzdanost):

- Korišćenje **ACID transakcija** (`prisma.$transaction`) prilikom prenosa novca i unosa troškova osigurava da podaci nikada ne budu u nekonzistentnom stanju (npr. da se novac skine sa jednog računa, a ne legne na drugi).

3. Performanse i Odziv:

- Filtriranje velike količine transakcija se vrši na klijentskoj strani (Client-side filtering u `TransactionHistory.tsx`), čime se smanjuje broj upita ka serveru i omogućava trenutni odziv interfejsa pri pretrazi.

5. Tehnička realizacija i DevOps postavka projekta

Ovo poglavlje opisuje implementaciju naprednih tehničkih zahteva, uključujući kontejnerizaciju, automatizaciju procesa integracije, zaštitu sistema i integraciju sa eksternim servisima, čime je aplikacija podignuta na produkcionu nivo.

5.1. Dockerizacija aplikacije

U cilju izolacije okruženja, lakšeg pokretanja i uniformnosti rada aplikacije na različitim operativnim sistemima, projekat je u potpunosti dockerizovan. Kontejnerizacija je realizovana pomoću **Docker** tehnologije.

U korenskom direktorijumu projekta kreiran je Dockerfile fajl koji definiše korake za kreiranje Docker slike za frontend i backend deo naše Next.js aplikacije. Ovaj fajl koristi optimizovanu Node.js bazu, instalira neophodne zavisnosti i vrši proces građenja aplikacije za produkciju.

Za orkestraciju kompletnog sistema koristi se **docker-compose** alat. Kreiran je docker-compose.yml fajl koji omogućava istovremeno pokretanje više kontejnera jednim klikom. U našem slučaju, definisana su dva glavna servisa:

1. **App servis:** Kontejner u kome se izvršava sama web aplikacija.
2. **Baza podataka (MySQL):** Zaseban kontejner koji sadrži bazu podataka, osiguravajući da podaci budu bezbedno smešteni i povezani sa aplikacijom preko definisanih portova.

Lokalno pokretanje kompletnog sistema, uključujući i bazu i aplikaciju, svodi se na izvršavanje jedne komande u terminalu:

```
docker-compose up -d
```

5.2. API specifikacija (Swagger)

Kako bi se olakšalo održavanje, testiranje i buduća integracija sa drugim sistemima, implementirana je automatska specifikacija API krajnjih tačaka (*endpoints*) korišćenjem **Swagger** alata (OpenAPI standard).

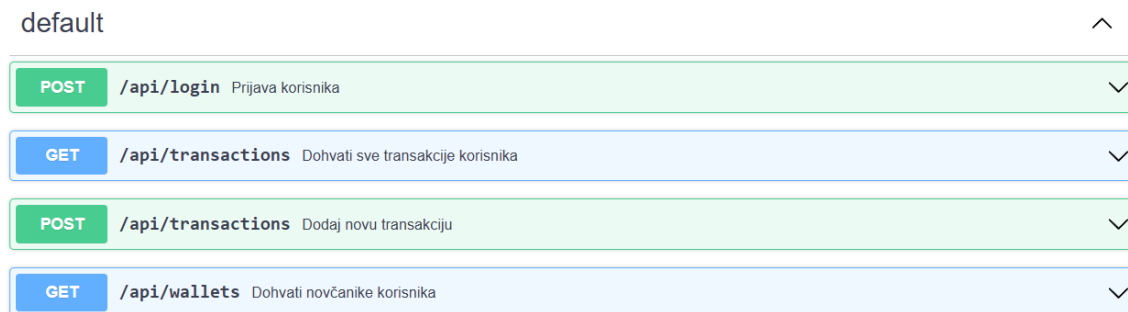
Dokumentacija je dinamički generisana i dostupna je preko posebne rute unutar aplikacije (/api-docs). Kroz korisnički interfejs Swagger UI-a, omogućen je jasan vizuelni pregled svih dostupnih metoda (GET, POST, PUT, DELETE) za različite resurse, kao što su /api/transactions, /api/wallets i /api/budgets.

Za svaku rutu jasno su definisani:

- Neophodni parametri zahteva (*request body*).
- Tipovi podataka koji se očekuju.
- Struktura JSON odgovora (*response*).
- Mogući statusni kodovi za uspešne operacije i obradu grešaka.

SmartBudget API 1.0.0 OAS 3.0

API dokumentacija za SmartBudget aplikaciju.



Slika 29 - Prikaz Swagger UI interfejsa sa automatski generisanom API dokumentacijom

5.3. Integracija sa eksternim API servisima

Kako bi se unapredilo korisničko iskustvo i pružile dodatne, relevantne informacije u realnom vremenu, u okviru početne strane (Dashboard) implementirana je komunikacija sa dva nezavisna spoljna (eksterna) web servisa. Komunikacija se vrši klijentski, asinhronim HTTP zahtevima ka javnim REST API-jima.

Za realizaciju ovog zahteva korišćeni su sledeći servisi:

1. ExchangeRate-API (Prikaz kursne liste)

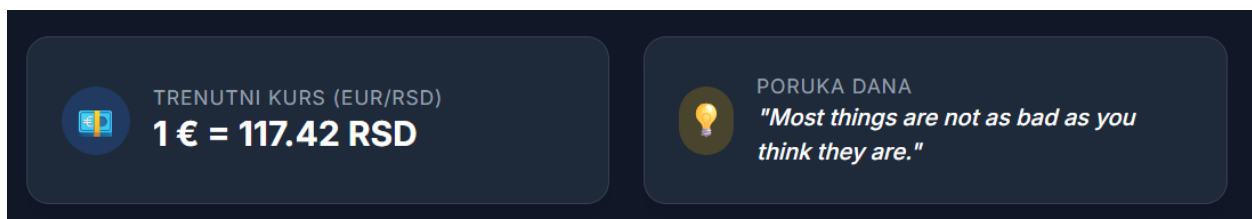
Ovaj API se koristi za preuzimanje aktuelnih kurseva valuta u realnom vremenu. S obzirom na to da korisnici upravljaju finansijama, prikazivanje odnosa ključnih svetskih valuta (poput EUR i USD) u odnosu na domaću valutu (RSD) predstavlja korisnu funkcionalnost. Aplikacija šalje GET zahtev ka zvaničnom endpoint-u servisa, a povratni JSON odgovor se parsira i prikazuje korisniku u obliku pregledne tabele ili vidžeta.

2. AdviceSlip API (Sistem za nasumične savete)

Kao dodatak koji obogaćuje korisnički interfejs, integrisan je AdviceSlip API. Ovaj servis na svaki upit vraća nasumičan, kratak savet ili motivacionu poruku. Iako generalne namene, u kontekstu aplikacije za budžetiranje služi kao element koji zadržava pažnju korisnika i čini interfejs dinamičnijim.

Tehnička implementacija:

Oba API poziva su realizovana unutar React komponente (ExternalWidgets.tsx). Za upravljanje asinhronim operacijama koriste se React hooks. Kuka useEffect se koristi za okidanje zahteva (putem standardne fetch funkcije) neposredno nakon montiranja komponente na ekran. Povučeni podaci se zatim smeštaju u lokalna stanja korišćenjem useState kuke. Pored uspešnog scenarija, implementirani su i try/catch blokovi za obradu grešaka, čime se osigurava da eventualni pad eksternog servera ne prouzrokuje prekid rada same aplikacije, već se korisniku prikazuje adekvatna poruka o nemogućnosti učitavanja podataka.



Slika 30 - Prikaz integrisanih eksternih API servisa na korisničkoj tabli

5.4. Dokumentacija projekta (README)

U korenskom direktorijumu repozitorijuma nalazi se README.md fajl koji služi kao osnovna tehnička dokumentacija izvornog koda. Ovaj dokument je strukturiran tako da omogućí brzo upoznavanje sa projektom i sadrži sledeće ključne informacije:

- **Opis aplikacije:** Na samom početku je dat jasan pregled namene sistema. SmartBudget je opisan kao moderna veb aplikacija za praćenje ličnih finansija, razvijena u okviru seminarskog rada. Navedene su glavne funkcionalnosti koje aplikacija pruža korisnicima, uključujući upravljanje novčanicima, praćenje prihoda i rashoda, postavljanje mesečnih budžeta, transfer novca i vizuelizaciju potrošnje.
- **Tehnologije koje su korišćene:** Precizno je izlistan celokupan tehnološki stek podeljen po slojevima arhitekture. Za razvoj korisničkog interfejsa (Frontend) navedeni su Next.js (React), Tailwind CSS i biblioteka Recharts za vizuelizaciju. Backend se oslanja na Next.js API rute uz poštovanje REST konvencija. Za perzistenciju podataka koriste se MySQL baza i Prisma ORM, dok je celokupna infrastruktura bazirana na Docker kontejnerima.
- **Instrukcije za lokalno pokretanje aplikacije:** Uputstvo detaljno vodi kroz proces podešavanja razvojnog okruženja. Prvo su definisani preduslovi koje računar mora ispuniti, što obuhvata instaliran Node.js (verzija 18+), Docker, Docker Desktop i Git. Zatim su priložene tačne komande za kloniranje repozitorijuma sa GitHub platforme (git clone) i pozicioniranje unutar foldera projekta.
- **Način pokretanja pomoću Docker-a i docker-compose-a:** Proces pokretanja celokupnog sistema je automatizovan i maksimalno pojednostavljen zahvaljujući kontejnerizaciji. U fajlu je objašnjeno da se aplikacija i baza podataka podižu istovremeno izvršavanjem jedne komande u terminalu:

```
docker-compose up -d --build
```

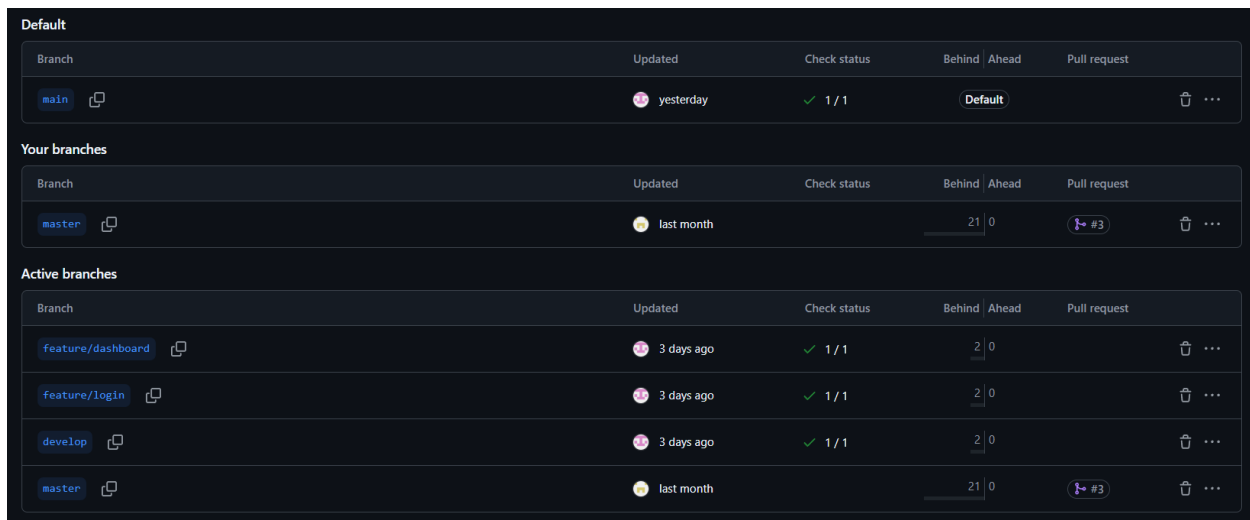
Takođe je navedeno da, nakon uspešnog građenja i pokretanja kontejnera, korisnik može pristupiti aplikaciji putem pretraživača na adresi *http://localhost:3000*. Na samom kraju fajla potpisani su autori projekta.

5.5. Dokumentacija projekta (README)

Primenjena je standardna strategija grananja (Git Flow), koja omogućava paralelni rad na različitim delovima aplikacije bez narušavanja stabilnosti produkcionog koda.

Repozitorijum je organizovan kroz sledeću strukturu grana:

- **main (stabilna produkciona grana):** Ovo je glavna grana projekta koja sadrži proveren, testiran i potpuno funkcionalan kod. Predstavlja stabilnu verziju aplikacije koja je direktno povezana sa Vercel platformom. Svako spajanje novog koda u ovu granu automatski okida proces produkcionog deployment-a na Cloud okruženje.
- **develop (integraciona grana):** Služi kao centralno okruženje za aktivni razvoj projekta. Sve nove funkcionalnosti i ispravke grešaka se prvo spajaju u ovu granu. develop grana omogućava testiranje interakcije različitih komponenti pre nego što se celokupan, integrisan kod prebaci na main granu.
- **Feature grane (radne grane):** Za svaki pojedinačni, izolovani zadatak kreirane su privremene radne grane odvajanjem od develop grane. Po završetku razvoja, ove grane se spajaju nazad u develop putem Pull Request-ova. U projektu su se, između ostalih, izdvojile sledeće feature grane:
 - **feature/login:** Grana namenski kreirana za implementaciju sistema autentifikacije. Na ovoj grani razvijane su forme za registraciju i prijavu, integracija Bcrypt heširanja lozinki i postavljanje sigurnosnih JWT sesija.
 - **feature/dashboard:** Radna grana fokusirana isključivo na vizuelni i funkcionalni razvoj glavne korisničke table. U okviru nje implementirana je integracija Recharts biblioteke za iscrtavanje grafikona, kao i povezivanje sa eksternim API servisima za prikaz kursne liste.



The screenshot displays the GitHub repository's branch structure, organized into three sections: Default, Your branches, and Active branches. Each section contains a table with columns for Branch, Updated, Check status, Behind/Ahead, and Pull request.

Default				
Branch	Updated	Check status	Behind / Ahead	Pull request
main	yesterday	✓ 1 / 1	Default	🗑️ ...

Your branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
master	last month		21 0	🔗 #3 🗑️ ...

Active branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
feature/dashboard	3 days ago	✓ 1 / 1	2 0	🗑️ ...
feature/login	3 days ago	✓ 1 / 1	2 0	🗑️ ...
develop	3 days ago	✓ 1 / 1	2 0	🗑️ ...
master	last month		21 0	🔗 #3 🗑️ ...

Slika 31 - Prikaz strukture grana na GitHub repozitorijumu

5.6. CI/CD pipeline

Za potrebe automatizacije procesa testiranja i isporuke softvera, u projektu je implementiran CI/CD pipeline korišćenjem platforme **GitHub Actions**. Ovaj mehanizam osigurava stabilnost koda pre svakog spajanja i automatizuje produkcionu deployment.

Celokupna konfiguracija nalazi se u korenskom direktorijumu repozitorijuma, unutar fajla `.github/workflows/main.yml`.

- **Pokretanje poslova (Push i Pull Request):** Pipeline je konfigurisan tako da se automatski okida (*trigger*) na svaki push događaj, kao i prilikom otvaranja pull request-a prema glavnim granama (main i develop). Sastoji se od dva glavna posla (*jobs*): test (za validaciju logike) i build (za validaciju kontejnerizacije).
- **Izvršavanje testova:** Prvi posao koji se pokreće jeste testiranje. GitHub Actions podiže izolovano Ubuntu virtuelno okruženje, preuzima najnoviji kod, instalira sve potrebne Node.js pakete (komandom `npm install`) i pokreće automatizovane Jest testove (komandom `npm run test`). Ukoliko bilo koji od testova padne, izvršavanje pipeline-a se prekida, a spajanje koda (*merge*) se blokira, čime se sprečava prenos grešaka u produkciju.
- **Građenje Docker image-a:** Nakon uspešnog prolaska svih testova, pokreće se sledeći posao u nizu – provera Docker konfiguracije. Sistem pokušava da izgradi (*build*) Docker image na osnovu instrukcija iz Dockerfile dokumenta. Ovaj korak je ključan jer potvrđuje da je aplikacija spremna za kontejnerizaciju i da nema sistemskih grešaka koje bi sprečile pokretanje na serveru.
- **Automatski deployment na Cloud:** Dok GitHub Actions služi za CI deo (kontinuiranu integraciju i testiranje), CD deo (kontinuirana isporuka) je rešen automatskom integracijom repozitorijuma sa **Vercel** platformom. Vercel je direktno povezan sa main granom. Čim se procesi u GitHub Actions uspešno završe i kod se spoji u main granu, Vercel detektuje promenu, automatski povlači najnoviji kod, prevodi ga (*build*) i postavlja uživo na Cloud produkciono okruženje bez potrebe za ručnom intervencijom.

- Isečak iz konfiguracije pipeline-a (main.yml):

```
name: SmartBudget CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

- main
- master
- develop

```
  pull_request:
```

```
    branches:
```

- main
- master
- develop

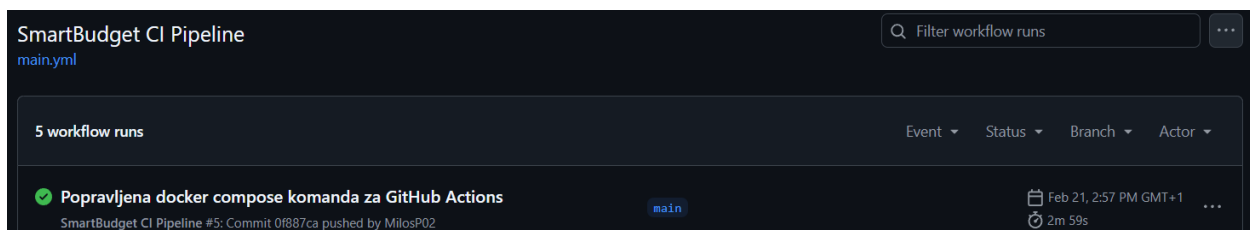
```
jobs:
```

```
  testiranje-i-build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Preuzimanje koda
 uses: actions/checkout@v4
- name: Podesavanje Node.js okruzenja
 uses: actions/setup-node@v4
 with:
 node-version: '20'
- name: Instalacija zavisnosti
 run: npm install --legacy-peer-deps
- name: Pokretanje automatizovanih testova
 run: npm run test
- name: Testiranje Docker Build-a
 run: docker compose build



Slika 32 - Prikaz uspešnog izvršavanja GitHub Actions pipeline-a

5.7. Postavljanje aplikacije na Cloud

U cilju omogućavanja javnog pristupa i testiranja aplikacije u realnom okruženju, sistem je uspešno postavljen (deploy-ovan) na Cloud infrastrukturu. S obzirom na specifičnu arhitekturu aplikacije, za hosting su korišćene dve različite platforme:

- **Baza podataka (Aiven):** Kao rešenje za perzistenciju podataka odabrana je platforma Aiven, na kojoj je kreirana namenska produkciona MySQL baza podataka u oblaku. Ovo rešenje osigurava visoku dostupnost, skalabilnost i bezbednost podataka.
- **Web aplikacija (Vercel):** Frontend korisnički interfejs i Backend (Next.js API rute) hostovani su na platformi Vercel, koja je preporučena i maksimalno optimizovana za rad sa Next.js okvirom.

Proces deploy-ovanja: Proces postavljanja aplikacije je u potpunosti automatizovan, čime je zaokružen CI/CD ciklus. Vercel platforma je direktno integrisana sa GitHub repozitorijumom projekta i konfigurisana da prati promene na glavnoj, produkcionoj main grani.

Konekcioni string produkcione baze podataka sa Aiven platforme, kao i drugi tajni ključevi, bezbedno su smešteni u okviru sistemskih promenljivih okruženja (Environment Variables) unutar Vercel platforme, čime se štiti bezbednost i integritet sistema. Kada se novi kod uspešno testira i spoji (merge) u main granu, Vercel samostalno i automatski detektuje promenu, inicira preuzimanje koda, instalira potrebne pakete, izvršava produkциони proces prevođenja (build) i isporučuje novu verziju aplikacije korisnicima. Ovaj mehanizam eliminiše potrebu za ručnom intervencijom i osigurava minimalno vreme zastoja.

Produkciona verzija aplikacije: Aplikacija je javno dostupna i može se testirati putem sledećeg linka:

<https://internet-tehnologije-2025-smartbudget-2021-0100-f8mbub06q.vercel.app>

5.8. Bezbednost aplikacije

Prilikom razvoja SmartBudget aplikacije, posebna pažnja posvećena je zaštiti korisničkih podataka i prevenciji najčešćih bezbednosnih propusta na vebu. Sistem primenjuje višeslojnu zaštitu (na nivou baze, servera i klijenta), a projekat je zaštićen od napada:

1. Zaštita od SQL Injection (SQLi) napada

- **Opis napada:** SQL injekcija je tehnika napada u kojoj zlonamerni korisnik unosi maliciozne SQL komande kroz ulazna polja na formama, sa ciljem manipulacije, brisanja ili krađe podataka direktno iz baze.
- **Način implementacije zaštite:** U projektu je direktna komunikacija sa MySQL bazom u potpunosti izbegnuta. Umesto pisanja sirovih SQL upita, arhitektura koristi **Prisma ORM** (Object-Relational Mapping). Prisma nativno i automatski vrši parametrizaciju i sanitizaciju svih podataka koji dolaze od korisnika pre nego što ih pošalje u bazu. Na ovaj način se svaki unos tretira isključivo kao obična vrednost, a nikada kao izvršna SQL komanda, čime je SQL Injection u potpunosti onemogućen.

2. Zaštita od Cross-Site Scripting (XSS) napada

- **Opis napada:** XSS napad se dešava kada napadač uspe da ubaci zlonamerni JavaScript kod u veb stranicu, koji se zatim izvršava u pregledaču drugih korisnika, omogućavajući krađu sesija ili preusmeravanje.
- **Način implementacije zaštite:** Frontend aplikacije je izgrađen na React i Next.js tehnologijama. React po zadatim postavkama vrši automatsko eskapiranje (escaping) svih promenljivih koje se renderuju kroz JSX sintaksu. To znači da, ukoliko korisnik u polje za naziv novčanika ili transakcije unese `<script>maliciozni kod</script>`, React to neće interpretirati kao HTML/JS kod, već će ga pretvoriti u običan, bezopasan tekst pre prikaza u DOM-u (Document Object Model).

3. Zaštita od Insecure Direct Object Reference (IDOR) i neovlašćenog pristupa

- **Opis napada:** IDOR je ranjivost koja nastaje kada aplikacija omogućava direktan pristup objektima u bazi (npr. transakcijama ili budžetima) na osnovu korisničkog unosa (poput ID-a u URL-u), bez adekvatne provere vlasništva nad tim objektom.
- **Način implementacije zaštite:** Na svakoj zaštićenoj API ruti na backendu vrši se striktna provera autorizacije i vlasništva. Kada korisnik pokuša da obriše ili izmeni transakciju šaljući DELETE/PUT zahtev na `/api/transactions/[id]`, sistem prvo validira sesiju korisnika i izvlači njegov `userId`. Zatim se vrši provera u bazi da li resurs sa prosleđenim ID-jem zaista pripada ulogovanom korisniku. Pored toga, primenjen je mehanizam kontrole uloga (Role-Based Access Control), gde je pristup osetljivim rutama (poput blokiranja korisnika) dozvoljen isključivo entitetima koji u bazi imaju postavljen status `role === 'ADMIN'`.

5.9. Automatizovani testovi

U cilju osiguravanja visokog kvaliteta koda i prevencije regresionih grešaka (bagova) prilikom uvođenja novih funkcionalnosti, u projekat su integrisani automatizovani testovi. Fokus testiranja je na kritičnim komponentama korisničkog interfejsa i logici aplikacije.

- **Vrste implementiranih testova i alati:** U okviru projekta implementirani su **jedinični (unit) testovi**. Za njihovo izvršavanje koristi se **Jest**, jedan od najpopularnijih JavaScript okvira za testiranje, u kombinaciji sa **React Testing Library** bibliotekom. Ovi alati omogućavaju simulaciju ponašanja korisnika u pregledaču i proveru ispravnosti renderovanja komponenti. Testovi su fokusirani na frontend deo aplikacije, konkretno na višekratne (reusable) UI komponente koje se koriste kroz ceo sistem.
- **Primeri testova:** Kao primer implementacije može se navesti test fajl `Button.test.tsx`. Ovaj test proverava da li se komponenta dugmeta ispravno renderuje sa zadatim tekstom, kao i da li se funkcija dodeljena na `onClick` događaj pravilno poziva prilikom klika. Ovakvim testovima se garantuje da osnovni elementi interfejsa reaguju na korisničke akcije onako kako je definisano poslovnom logikom.
- **Pokretanje testova lokalno:** Programer može pokrenuti sve testove u lokalnom okruženju jednostavnom komandom unutar terminala:

```
npm run test
```

Ova komanda pokreće Jest pokretač koji skenira projekat, pronalazi sve fajlove sa ekstenzijom `.test.tsx` i prikazuje detaljan izveštaj o tome koji su testovi prošli, a koji nisu.

- **Pokretanje testova kroz CI:** Kao što je opisano u poglavlju o CI/CD pipeline-u, testovi su integralni deo automatizovanog procesa na GitHub platformi. Na svaki push ili pull request, GitHub Actions automatski podiže virtuelno okruženje i izvršava istu `npm run test` komandu. Ukoliko testovi ne prođu, proces integracije se obustavlja, čime se onemogućava da neispravan kod stigne do produkcionog servera.

Isečak koda iz `__tests__/Button.test.tsx`:

```
import "@testing-library/jest-dom";
import { render, screen, fireEvent } from "@testing-library/react";
import Button from "../src/components/ui/Button";

describe("Button komponenta", () => {
  it("treba da prikaže tačan tekst koji mu se prosledi", () => {
    render(<Button>Klikni me</Button>);

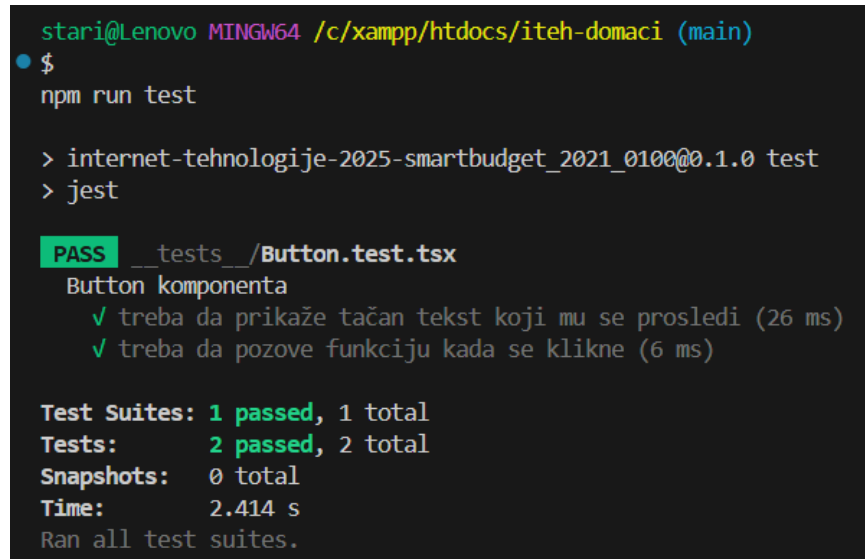
    const buttonElement = screen.getByText("Klikni me");
    expect(buttonElement).toBeInTheDocument();
  });

  it("treba da pozove funkciju kada se klikne", () => {
    const mockOnClick = jest.fn();

    render(<Button onClick={mockOnClick}>Test Dugme</Button>);

    const buttonElement = screen.getByText("Test Dugme");
    fireEvent.click(buttonElement);

    expect(mockOnClick).toHaveBeenCalledTimes(1);
  });
});
```



```
stari@Lenovo MINGW64 /c/xampp/htdocs/iteh-domaci (main)
$ npm run test

> internet-tehnologije-2025-smartbudget_2021_0100@0.1.0 test
> jest

PASS __tests__/Button.test.tsx
  Button komponenta
    ✓ treba da prikaže tačan tekst koji mu se prosledi (26 ms)
    ✓ treba da pozove funkciju kada se klikne (6 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.414 s
Ran all test suites.
```

Slika 33 - Prikaz uspešnog izvršavanja automatizovanih testova u lokalnom okruženju

5.10. Vizuelizacija podataka

U okviru glavne kontrolne table (Dashboard) implementirana je funkcionalnost vizuelizacije podataka kako bi se korisnicima omogućio jasan i intuitivan pregled njihovih finansijskih tokova. Umesto suvoparnog tabelarnog prikaza, korisnik može grafički analizirati svoje prihode i rashode.

- **Podaci koji se vizuelizuju:** Aplikacija vrši vizuelizaciju kumulativnih finansijskih transakcija. Glavni fokus je na praćenju odnosa ukupnih prihoda i ukupnih rashoda korisnika, kao i na pregledu stanja po različitim kategorijama troškova. Ovi podaci pomažu korisniku da brzo identifikuje na šta troši najviše novca i kakav je njegov mesečni balans.
- **Alati i vrste grafikona:** Za generisanje prikaza korišćena je **Recharts** biblioteka, koja je bazirana na React-u i omogućava kreiranje responzivnih grafikona. U aplikaciji su primenjene dve vrste prikaza:
 1. **Stubični dijagram (Bar Chart):** Koristi se za direktno poređenje ukupnih prihoda i rashoda, gde visina stubaca jasno ukazuje na finansijski odnos.
 2. **Kružni dijagram (Pie Chart):** Koristi se za prikaz raspodele troškova po kategorijama (npr. hrana, stanarina, zabava), gde svaki isečak kruga predstavlja udeo određene kategorije u ukupnoj potrošnji.
- **Preuzimanje podataka iz API-ja:** Podaci se preuzimaju asinhrono sa backenda putem interne API rute `/api/transactions`. Unutar komponente `DashboardCharts.tsx`, koristi se React kuka `useEffect` koja šalje HTTP GET zahtev ka serveru. Dobijeni JSON odgovor, koji sadrži listu svih transakcija, zatim se pomoću JavaScript funkcija (`reduce` i `map`) transformiše u format koji Recharts biblioteka zahteva (niz objekata sa definisanim imenima i vrednostima).



Slika 34 - Grafički prikaz finansijskih statistika na korisničkoj tabli